

Vectorization

And compilers

Introduction

- Perhaps, together with the operating system, the compiler is THE most important HPC software tool.
- Compilers are very sophisticated software tools but cannot replace human understanding of what the code should do.
 - They can attempt to optimise the code.
- There are many compilers available and for all computer operating systems, both free and commercial
- The Intel Compiler
<https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference>

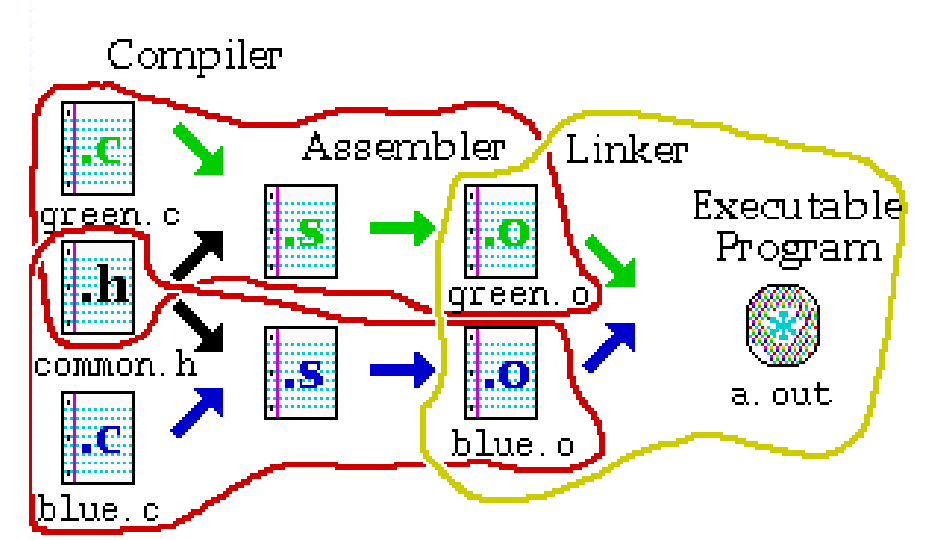
Compilers and interpreters

- Interpreted languages
 - The code is “translated” statement-by-statement during the execution
 - Easier on the programmer, modifications can be made quickly but optimisations between different statements (almost) impossible
 - Used for scripting languages (bash, Perl, PHP, ..)
- Compiled languages
 - Entire program is translated before execution
 - Optimizations between different parts of the program possible.
 - HPC languages include FORTRAN, C and C++
 - Python? https://sea.ucar.edu/sites/default/files/python_hpc.pdf
http://www.training.prace-ri.eu/uploads/tx_pracetmo/pythonHPC.pdf

Compilers stages

“Compiling” a program is actually a three stage process:

1. Pre-processing to replace MACROs (**#define**), code insertions (**#include**), code selections (**#ifdef**, **#if**).
2. Compilation of the source code into object files – organised collections of symbols referring to variables and functions.
 - <https://www.howtoforge.com/linux-nm-command/>
 - Code optimisations are mainly done during compilation.
3. Linking of the object files, together with any external libraries to create the executable (if all referred objects are resolved).
 - For large projects usual to separate the compiling and linking phases.



Common compiler options – icc

Functionality	Linux
Disable optimization	-O0
Optimize for speed (no code size increase)	-O1
Optimize for speed (default)	-O2
High-level optimizer (e.g. loop unroll)	-O3
Aggressive optimizations (e.g. -ipo, -O3, ...)	-fast
Create symbols for debugging	-g
Generate assembly files	-S
Optimization report generation	-opt-report
OpenMP support	-openmp

Functionality	Linux *
Optimize for current machine	-xHOST
Generate SSE v1 code	-xSSE1
Generate SSE v2 code (default, may also emit SSE v1 code)	-xSSE2
Generate SSE v3 code (may also emit SSE v1 and v2 code)	-xSSE3
Generate SSE v3 code for Atom-based processors	-xSSE_ATOM
Generate SSSE v3 code (may also emit SSE v1, v2, and v3 code)	-xSSSE3
Generate SSE4.1 code (may also emit (S)SSE v1, v2, and v3 code)	-xSSE4.1
Generate SSE4.2 code (may also emit (S)SSE v1, v2, v3, and v4 code)	-xSSE4.2
Generate AVX code	-xAVX

* For Intel processors use -x, for non-Intel processors use -m

<https://software.intel.com/content/www/us/en/develop/documentation/cpp-compiler-developer-guide-and-reference/top/compiler-reference/compiler-options/alphabetical-list-of-compiler-options.html>

Source code and performance

- <https://www.intel.com/content/www/us/en/search.html?ws=text#q=code%20sample&t=Developers&sort=relevancy>
- <https://github.com/intel-system-studio/samples>
Do in your home git clone <https://github.com/intel-system-studio/samples.git>
- Local copy of files /opt/intel/composer_xe_2013.5.192/composer_xe_2013.5.192/Samples/en_US/C++ ,
/opt/intel/samples_2019/en/ /opt/intel/samples_2017/en/ ...

Example

- <https://software.intel.com/en-us/advisor-tutorial-vectorization-linux-cplusplus-simd-parallelism-use-case>
- `icc -std=c99 -O0 -D NOFUNCCALL Multiply.c Driver.c -o MatVector` (*do not cut&paste the commands, rewrite them*)
- Let's try also O1 and O3

O0

ROW:101 COL: 101

Execution time is 331.700 seconds

GigaFlops = 0.061507

Sum of result = 195853.999899

O1

ROW:101 COL: 101

Execution time is 49.423 seconds

GigaFlops = 0.412806

Sum of result = 195853.999899

O2/O3

ROW:101 COL: 101

Execution time is 12.435 seconds

GigaFlops = 1.640643

Sum of result = 195853.999899

Compiler optimization

- Compilers give the possibility of specifying optimisation options at compile time, together with the other options.
 - -O0 : no optimisation, the code is translated literally
 - -O1, -O2: local optimisations, compromise between compilation speed, optimisation, code accuracy and executable size (in icc O2 is default, see icc --help)
 - -O3: high optimisation, can alter the semantics of the program (hence not used for debugging)
 - -O4 or higher: Aggressive optimisations, depending on hardware.
- These are either general optimisation levels or specific flags related to the underlying hardware.
- Compilers are generally conservative, i.e. will not optimise if strong risk of obtaining incorrect results *unless* forced to by the user either with compiler directives or options.

Pay attention

Increasing the optimisation level doesn't always increase performance. Must check each time!

- Example: matrix-matrix multiplication (1024x1024), double precision, FORTRAN.
- Two systems:
 - FERMI: (IBM BG/Q Power A2, 1.6Ghz)
 - PLX: (Xeon Westmere CPUs, 2.4 Ghz)

The compiler recognises the matrix-matrix product and substitutes the code with a call to a library routine `__xl_dgemm`, but it is quite slow. Intel on PLX uses a similar strategy, but uses instead the efficient MKL library

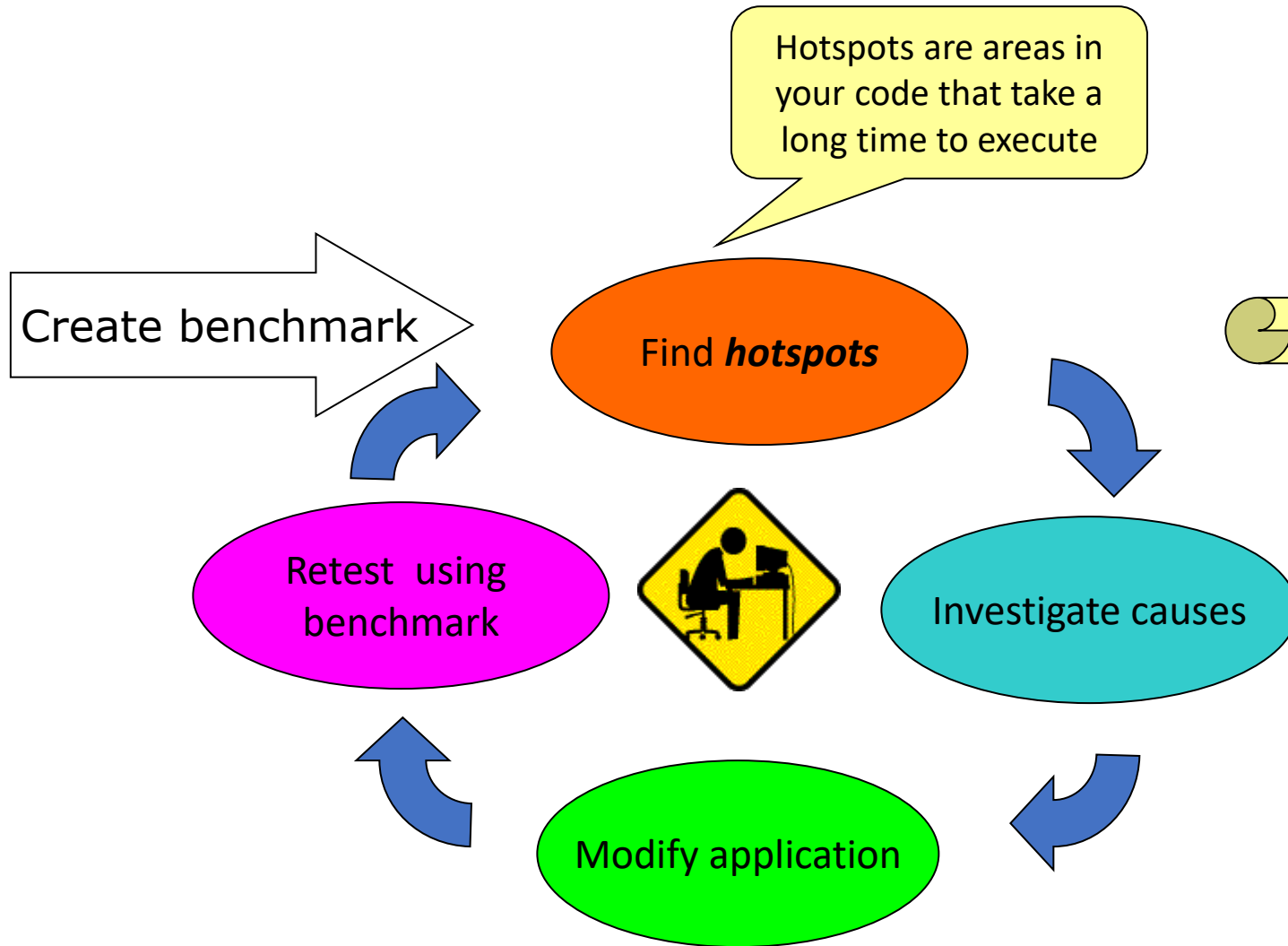
FERMI xlf

Option	Seconds	MFlops
-O0	65.78	32.6
-O2	7.13	301
-O3	0.78	2735
-O4	55.52	38.7
-O5	0.65	3311

PLX -ifort

Option	Seconds	MFlops
-O0	8.94	240
-O2	1.41	1514
-O3	0.72	2955
-O4	0.33	6392
-O5	0.32	6623

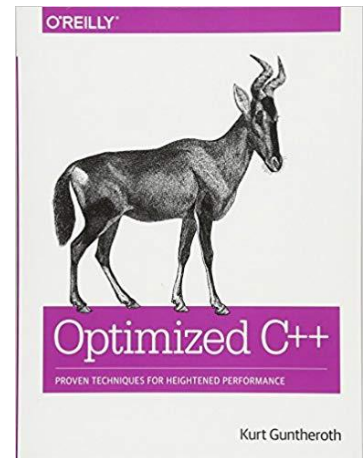
The Software Optimization Process



Software optimization does not begin where coding ends. It is an ongoing process that starts at design stage and continues all the way through development.

The Software Optimization Facts

- It's easier to optimize a slow correct program than to debug a fast incorrect one
 - *Nobody cares how fast you can compute a wrong answer...*
- Programs typically spend 80% of their time in 20% of the code
 - Programmers typically spend 20% of their effort to get 80% of the total speedup possible for the application
 - *Know when to stop!*
- Don't optimize what does not matter
 - *Make the common case fast!*



1.
Optimizing software in C++
An optimization guide for Windows, Linux, and Mac
platforms

By Agner Fog. Technical University of Denmark.
Copyright © 2004 - 2019. Last updated 2019-09-14.

A first optimization: the Vectorization

- From Intel: it is the unrolling of a loop combined with the generation of packed SIMD instructions by the compiler.
- Because the packed instructions operate on more than one data element at a time, the loop can execute more efficiently.
- It is sometimes referred to as auto-vectorization, to emphasize that the compiler identifies and optimizes suitable loops on its own, without requiring any special action by the programmer.

<https://www.intel.com/content/dam/develop/external/us/en/documents/31848-compilerautovectorizationguide.pdf>

<https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference-programming-guidelines-for-vectorization>

<https://software.intel.com/sites/default/files/m/4/8/8/2/a/31848-CompilerAutovectorizationGuide.pdf>

<https://software.intel.com/content/dam/develop/external/us/en/documents/dpd-vectorization-codebook-558870.pdf>

Recap

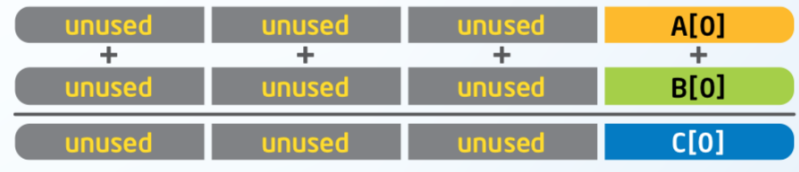
Scalar version works on one element at a time

$$a[i] = b[i] + c[i] \times d[i];$$

Vector version carries out the same instructions on many elements at a time

$$a[i:8] = b[i:8] + c[i:8] \times d[i:8];$$

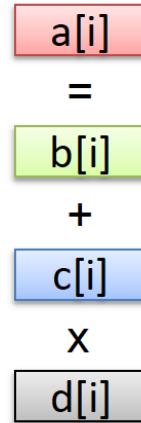
Scalar Implementation



```
for (i=0; i<N;i++) {  
    a[i]=b[i]+c[i];  
}
```



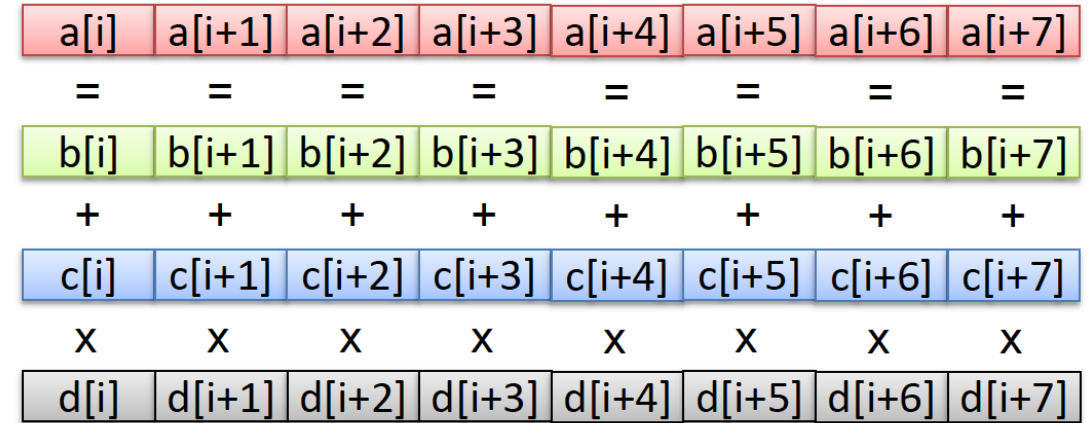
```
for (i=0; i<N;i+=4) {  
    a[i+0]=b[i+0]+c[i+0];  
    a[i+1]=b[i+1]+c[i+1];  
    a[i+2]=b[i+2]+c[i+2];  
    a[i+3]=b[i+3]+c[i+3];  
}
```



Load b(i..i+3)
Load c(i..i+3)
Operate b+c->a 4 assembly ops wrt 4*4
Store a



Made by the compiler – avoid to unroll loops by yourself



Loop unrolling

- Loop unrolling, also known as loop unwinding, is a loop transformation technique that attempts to optimize a program's execution speed at the expense of its binary size, which is an approach known as space–time tradeoff. The transformation can be undertaken manually by the programmer or by an optimizing compiler.
- The goal of loop unwinding is to increase a program's speed by reducing or eliminating instructions that control the loop, such as pointer arithmetic and "end of loop" tests on each iteration; reducing branch penalties; as well as hiding latencies, including the delay in reading data from memory. To eliminate this computational overhead, loops can be re-written as a repeated sequence of similar independent statements.

https://en.wikipedia.org/wiki/Loop_unrolling

An example using RISC-V assembler

The DAXPY cycle $Y=a*X+Y$, where X and Y are vectors ($Y[i]=a*X[i]+Y[i]$)

If AVX512 and X and Y have 8 elements, 66 ops wrt 6, speedup $11 > 8$!

Conventional processor

```
fld f0, a(x3)          // load scalar a
addi x5, x19, 512       // end of array X
loop: fld f1, 0(x19)    // load x[i]
fmul.d f1, f1, f0       // a * x[i]
fld f2, 0(x20)          // load y[i]
fadd.d f2, f2, f1       // a * x[i] + y[i]
fsd f2, 0(x20)          // store y[i]
addi x19, x19, 8        // increment index to x
addi x20, x20, 8        // increment index to y
bltu x19, x5, loop    // repeat if not done
```

Vectorised version

```
fld f0, a(x3)          // load scalar a
fld.v v0, 0(x19)       // load vector x
fmul.d.vs v0, v0, f0    // a * X
fld.v v1, 0(x20)       // load vector y
fadd.d.v v1, v1, v0     // vector-vector add
fsd.v v1, 0(x20)       // store vector y
```

How to achieve this goal?

```
__m128i add4(__m128i a, __m128i b) {  
    return _mm_add_epi32(a, b);  
}
```

- Let the compiler vectorize automatically is THE best choice (at least O2)
- But alternative strategies are

1. Write sections of a program in assembly language

<https://www.codeproject.com/Articles/15971/Using-Inline-Assembly-in-C-C>

2. Use higher level vector intrinsics

<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>

<https://www.intel.com/content/www/us/en/develop/documentation/cpp-compiler-developer-guide-and-reference/top/compiler-reference/intrinsics/details-about-intrinsics.html>

https://chryswoods.com/vector_c++/immintrin.html (an example)

You obviously can do nothing if you have time and resources to waste...

```
#include <stdio.h>  
  
int main() {  
    /* Add 10 and 20 and store result into register %eax */  
    __asm__ ( "movl $10, %eax;"  
              "movl $20, %ebx;"  
              "addl %ebx, %eax;"  
            );  
  
    /* Subtract 20 from 10 and store result into register %eax */  
    __asm__ ( "movl $10, %eax;"  
              "movl $20, %ebx;"  
              "subl %ebx, %eax;"  
            );  
  
    /* Multiply 10 and 20 and store result into register %eax */  
    __asm__ ( "movl $10, %eax;"  
              "movl $20, %ebx;"  
              "imull %ebx, %eax;"  
            );  
  
    return 0 ;  
}
```

Alignment and Interprocedural optimizations

- The vectorizer can generate faster code when operating on aligned data. For example if you align the arrays a, b, and x in Driver.c on a 16-byte boundary the vectorizer can use aligned load instructions for all arrays rather than the slower unaligned load instructions and can avoid runtime tests of alignment.
- The compiler may be able to perform additional optimizations if it is able to **optimize across source line** boundaries. These may include function inlining.

```
icc -std=c99 -D NOFUNCCALL -D ALIGNED -ipo -O3 Multiply.c Driver.c -o MatVector
```

Execution time is 12.775 seconds - GigaFlops = 1.597076 (so , little or no improvements in this case)

The fastest execution? NO! Add **-xHost** to use the highest Instruction set available in every CPU (here AVX512). But the executable becomes less portable (only Linux systems with AVX512 cpus)

Execution time is 5.798 seconds - GigaFlops = 3.519013

Pay attention

-march= tells the compiler to generate code for processors that support certain features.

-x Tells the compiler which processor features it may target, including which instruction sets and optimizations it may generate.

Options -x and -m are mutually exclusive. If both are specified, the compiler uses the last one specified and generates a warning.

On Linux systems, if neither -x nor -m is specified, the default target architecture supports Intel(R) SSE2 instructions.

In case of AVX512 on a processor different from KNL it could be necessary to specify, instead, -xCORE-AVX512 | -xCOMMON-AVX512 AND -qopt-zmm-usage=high

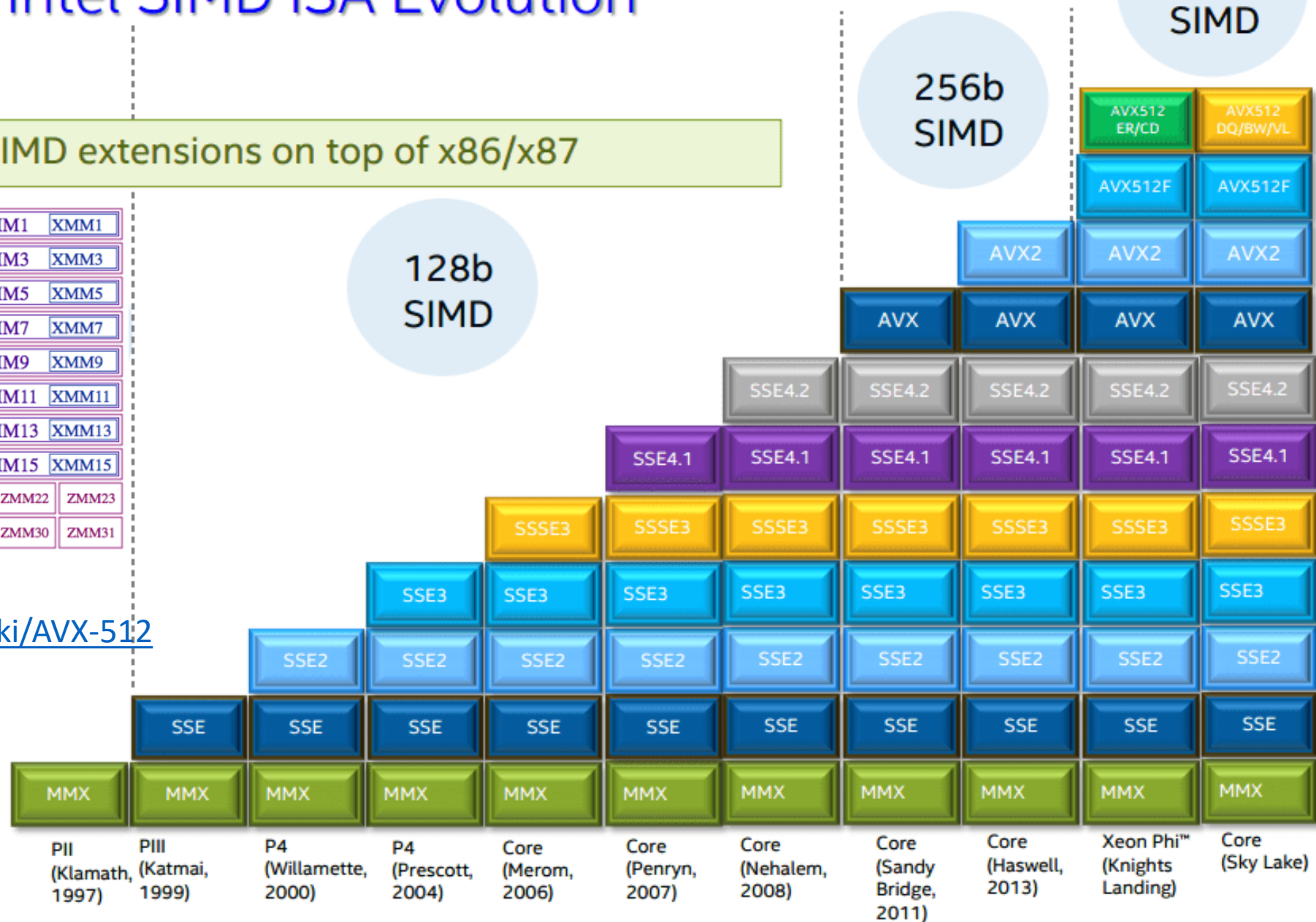
Recap

Intel SIMD ISA Evolution

SIMD extensions on top of x86/x87

ZMM0	YMM0	XMM0	ZMM1	YMM1	XMM1		
ZMM2	YMM2	XMM2	ZMM3	YMM3	XMM3		
ZMM4	YMM4	XMM4	ZMM5	YMM5	XMM5		
ZMM6	YMM6	XMM6	ZMM7	YMM7	XMM7		
ZMM8	YMM8	XMM8	ZMM9	YMM9	XMM9		
ZMM10	YMM10	XMM10	ZMM11	YMM11	XMM11		
ZMM12	YMM12	XMM12	ZMM13	YMM13	XMM13		
ZMM14	YMM14	XMM14	ZMM15	YMM15	XMM15		
ZMM16	ZMM17	ZMM18	ZMM19	ZMM20	ZMM21	ZMM22	ZMM23
ZMM24	ZMM25	ZMM26	ZMM27	ZMM28	ZMM29	ZMM30	ZMM31

<https://en.wikipedia.org/wiki/AVX-512>



Crash course in SIMD assembly

Register names

- SSE : xmm0 to xmm15 (128 bits)
- AVX2 : ymm0 to ymm15 (256 bits)
- AVX512 : zmm0 to zmm31 (512 bits)

In scalar mode, SSE registers are used

floating point instruction names

where `<op><simd or not><raw type>`

- `<op>` is something like `vmul`, `vadd`, `vmov` or `vfmadd`
- `<simd or not>` is either 's' for scalar or 'p' for packed (i.e. vector)
- `raw type` is either 's' for single precision or 'd' for double precision

Typically :

`vmulss, vmovaps, vaddpd, vfmaddpd`

Istantanea Schermo

Practical look at assembly

Extract assembly code

- Run `objdump -d -C` on your executable or library
- Search for your function name

Or use Intel Advisor

Check for vectorization

- For avx2, look for ymm
- For avx512, look for zmm
- Othersize look for instructions with ps or pd at the end
 - but ignore mov operations
 - only concentrate on arithmetic ones

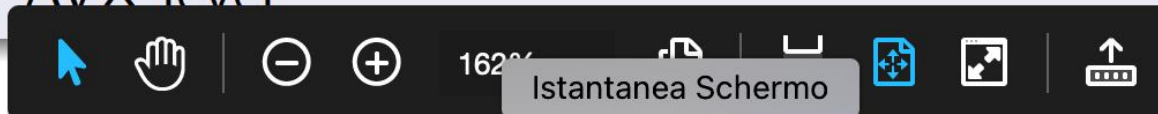
Exercise 1

Code

```
d18:  c5 fc 59 d8      vmulps %ymm0,%ymm0,%ymm3
d1c:  c5 fc 58 c0      vaddps %ymm0,%ymm0,%ymm0
d20:  c5 e4 5c de      vsubps %ymm6,%ymm3,%ymm3
d24:  c4 c1 7c 59 c0   vmulps %ymm8,%ymm0,%ymm0
d29:  c4 c1 64 58 da   vaddps %ymm10,%ymm3,%ymm3
d2e:  c4 41 7c 58 c3   vaddps %ymm11,%ymm0,%ymm8
d33:  c5 e4 59 d3      vmulps %ymm3,%ymm3,%ymm2
d37:  c4 c1 3c 59 f0   vmulps %ymm8,%ymm8,%ymm6
d3c:  c5 ec 58 d6      vaddps %ymm6,%ymm2,%ymm2
```

Solution

- Presence of ymm
- Vectorized, AVX level



Data alignment

- Data alignment is a method to force the compiler to create data objects in memory on specific byte boundaries.
- This is done to increase efficiency of data loads and stores to and from the processor.
- When using SSE intrinsics, you should align data to 16 bytes in memory operations. With AVX512 you should consider 64 bytes.
- Contiguous memory locations are loaded in an easy way, otherwise "by hand"

```
// a is 16-byte aligned  
float a[4] = {1.0, 2.0, 3.0, 4.0};  
__m128 t = _mm_load_ps(a);
```

p: packed
s: single

LSB

1.0	2.0	3.0	4.0
-----	-----	-----	-----

- Same result as

```
__m128 t = _mm_set_ps(4.0, 3.0, 2.0, 1.0)
```

How to align Static Arrays

- You can use an appropriate clause/attribute or option as described below to align the base-pointer.
- Once you do this, the data will be allocated at a good boundary AND the compiler will recognize alignment of the base-pointer for these arrays at all use sites.
- With Windows C/C++ `__declspec(align(64)) float A[1000];`
- With Linux/Mac C/C++ `float A[1000] __attribute__((aligned(64)));`

<https://www.intel.com/content/www/us/en/developer/articles/technical/data-alignment-to-assist-vectorization.html>

How to align Dynamic data

```
void myfunc2( double *p2, double *p3, double *p4, int n)
{
    for (int j=0; j<n; j+=8) {
        __assume_aligned(p2, 64);
        __assume_aligned(p3, 64);
        __assume_aligned(p4, 64);
        for (int i=0; i<8; i++) {
            p2[j+i] = p3[j+i] * p4[j+i];
        }
    }
}
```

- Replace malloc() and free() with alignment specified replacement _mm_malloc() and _mm_free().
- These routines take an alignment parameter (in bytes) as the second argument. These replacements provided by the Intel C++ Compiler also use the same size argument and return types as malloc() and free().
- The returned data will be 64 byte aligned.

buf = (char*) _mm_malloc(bufsizes[i], 64);

- Note that for C/C++ arrays that are dynamically allocated, **it is not enough**. It is also required to use a clause of the form `__assume_aligned(a, 64)` before the loop of interest.
- Without this step, the compiler will not detect the optimal alignment for accesses using such arrays. This is discussed more in a later section of this article.
- Different in GCC https://www.gnu.org/software/libc/manual/html_node/Aligned-Memory-Blocks.html

Intrinsics example with SSE

We assume properly aligned vectors

```
void sum_arrays(int *A, int *B, int *C, int len) {  
    for (int i = 0; i < len; ++i) C[i] = A[i] + B[i];  
}
```

*Solution simple if len = 4*something*

```
void sum_arrays(int *A, int *B, int *C, int len) {  
    for (int i = 0; i < len; i += 4) {  
        __m128i v1 = loadu_si128(&A[i]);  
        __m128i v2 = loadu_si128(&B[i]);  
        __m128i res = add4(v1, v2);  
        store_si128(&C[i], res);  
    }  
}
```

Pseudocode

```
void sum_arrays(int *A, int *B, int *C, int len) {  
    for (int i = 0; i < len; i += 4) {  
        v1 = load 4 values from A  
        v2 = load 4 values from B  
        res = 4-packed add(v1, v2)  
        store res (which is 4 values) in C  
    }  
}
```

Full solution

```
void sum_arrays(int *A, int *B, int *C, int len) {  
    int end = (len % 4 == 0) ? len : len - 4;  
    int i = 0;  
    for (; i < end; i += 4) {  
        __m128i v1 = loadu_si128(&A[i]);  
        __m128i v2 = loadu_si128(&B[i]);  
        __m128i res = add4(v1, v2);  
        store_si128(&C[i], res);  
    }  
    for (; i < len; ++i) C[i] = A[i] + B[i];  
}
```

BASIC requirements for Vectorization

Countable at runtime

- Number of loop iterations is known before loop executes (*i* does not change in the body)
- No conditional termination (break statements)

```
while (i < 100) {  
    a[i] = b[i] * c[i];  
    if (a[i] < 0.0) // data-dependent  
        break;  
    ++i;  
} // loop not vectorized
```

Have single control flow

- No switch statements
- 'if' statements are allowable when they can be implemented as masked assignments (e.g. if .. var = x else var = y)

Must be the innermost loop if nested

- The outermost loop is normally parallelized with openMP
- Compiler may reverse loop order as an optimization!

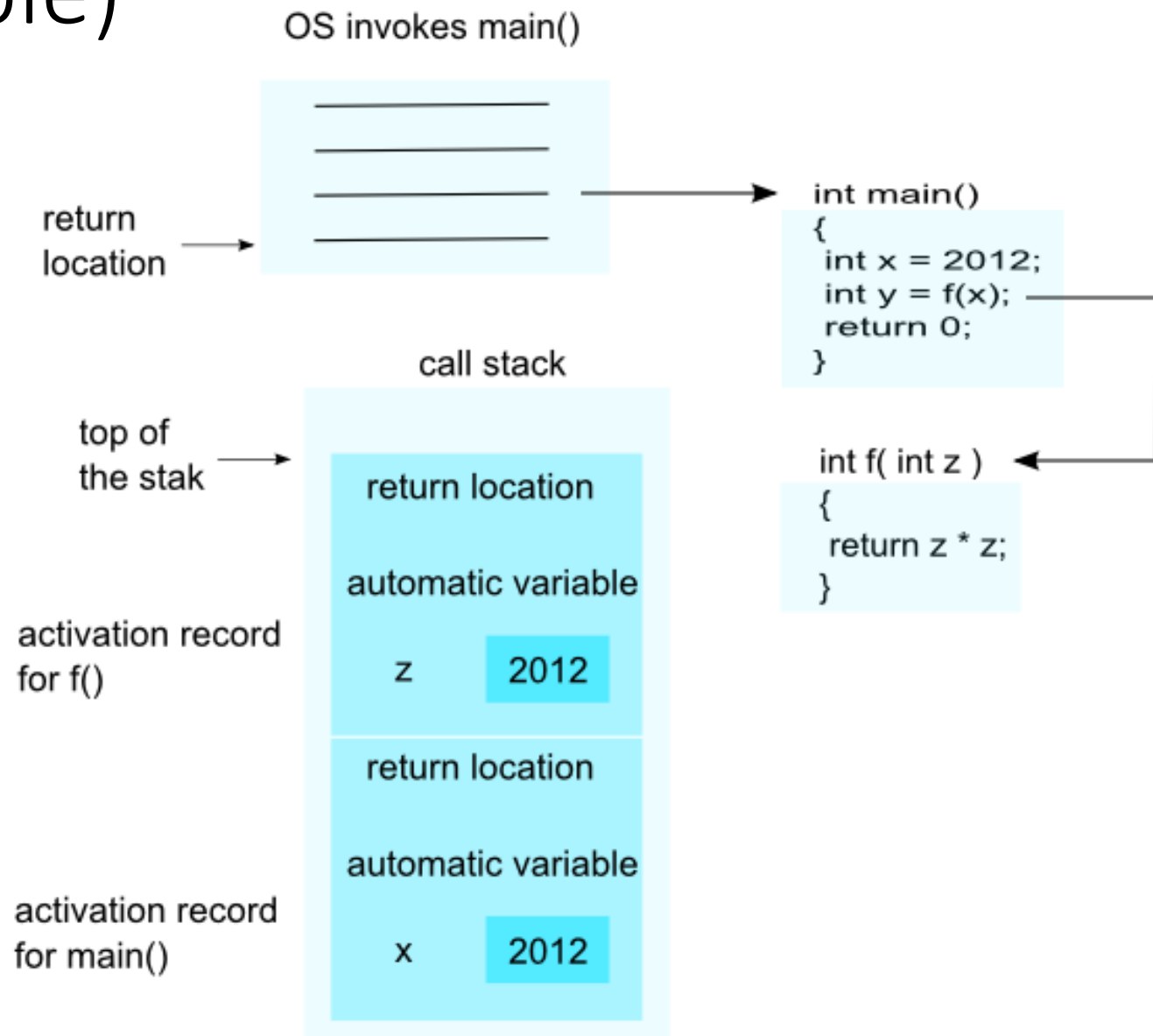
No function calls

- Basic math is allowed: pow(), sqrt(), sin(), etc
- Some inline functions allowed

```
for (int i = 0; i < length; i++) {  
    float s = b[i] * b[i] - 4 * a[i] * c[i];  
    if (s >= 0) x[i] = sqrt(s);  
    else x[i] = 0.;  
} // loop vectorized (because of masking)
```

NOFUNCCALL (example)

- Function calls can represent a major bottleneck during program execution since they usually force register contents to be spilled into memory.
- Such register to memory spills are much more pronounced in presence of recursion.
- A function call is usually accompanied by the creation of its activation record at function entry.



Vectorization Report

Shows which loops are or are not vectorized and why with `-qopt-report-phase=vec`
By default, reports are written to files suffixed with `.optrpt`, one per source file.

- Intel: `-qopt-report =<n>`
 - 0: None
 - 1: Lists vectorized loops
 - 2: 1 + Lists loops not vectorized, with explanation
 - 3: 2 + Outputs additional dependency information
 - 4: 3+ Lists loops not vectorized, without explanation
 - 5: All the available info
- Reports are essential for determining where the compiler finds a dependency
- Compiler is conservative, you need to go back and verify that there really is a dependency.
- Works also with `-qopt-report-phase=openmp` or `par`
- **Don't use `-ipo` when you ask the report!**

Details

icc -std=c99 -O3 -D NOFUNCCALL -qopt-report=2 -qopt-report-phase=vec Multiply.c Driver.c -o MatVector

```
LOOP BEGIN at Driver.c(145,2)
  remark #15542: loop was not vectorized: inner loop was already vectorized
LOOP BEGIN at Driver.c(148,3)
  remark #15542: loop was not vectorized: inner loop was already vectorized
  LOOP BEGIN at Driver.c(150,4)
    remark #15300: LOOP WAS VECTORIZED
  LOOP END
  LOOP BEGIN at Driver.c(150,4)
    <Remainder loop for vectorization>
  LOOP END
LOOP END
LOOP END
```

This is not true if you omit -D NOFUNCCALL

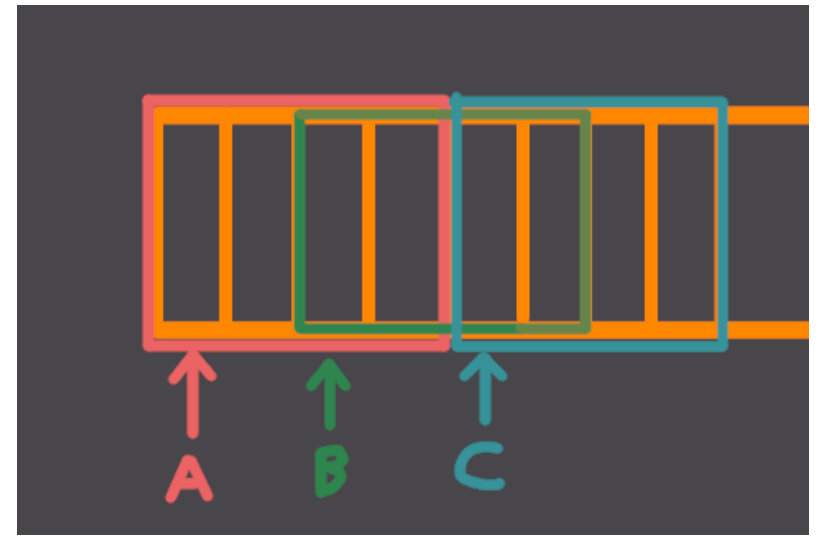
```
icc -std=c99 -O3 -qopt-report=2 -qopt-report-phase=vec
Multiply.c Driver.c -o MatVector
```

Execution time is 44.542 seconds
GigaFlops = 0.458044

```
143      //start timing the matrix multiply code
144      startTime = clock_it();
145      for (k = 0; k < REPEATNTIMES; k++) {
146  #ifdef NOFUNCCALL
147          int i, j;
148          for (i = 0; i < size1; i++) {
149              b[i] = 0;
150              for (j = 0; j < size2; j++) {
151                  b[i] += a[i][j] * x[j];
152              }
153          }
154  #else
155          matvec(size1, size2, a, b, x);
156  #endif
157          x[0] = x[0] + 0.000001;
158      }
159      endTime = clock_it();
160      execTime = endTime - startTime;
```

Aliasing and correctness

$C[i] = A[i] + B[i]$, but $B[0-1]$ is $A[2-3]$ and, more important, $C[0-1]$ is $B[2-3]$



- In the scalar version, we first write into $C[0]$ in the first iteration ($i==0$) and then we read from it (using $B[2]$) in the third iteration ($i==2$)
- In the vectorized version, the first iteration covers 0-4. We will read the old value of $B[2]$ (i.e. whatever was there before), before writing into it and that's obviously wrong.
- In these cases no way, the compiler cannot vectorialise. But if they do not overlap? We must say it "trust me, this pointer does not alias with any other pointer in the function"

Aliases

Two pointers are aliased if both point to the same memory location. Storing to memory using a pointer that might be aliased may prevent some optimizations.

Sometimes, the compiler can generate both a vectorized and a non-vectorized version of a loop and test for aliasing at runtime to select the appropriate code path.

If you know that pointers do not alias and **inform the compiler**, it can avoid the runtime check.

icc ... **-DNOALIAS** -xHost Multiply.c Driver.c
-o MatVector

Execution time is 22.623 seconds
GigaFlops = 0.901823

Vs (**#else** branch)

Execution time is 45.749 seconds
GigaFlops = 0.445955

```
LOOP BEGIN at Multiply.c(55,3)
  remark #15344: loop was not vectorized: vector dependence prevents vectorization.
First dependence is shown below. Use level 5 report for details
  remark #15346: vector dependence: assumed FLOW dependence between b[i] (56:4) and
b[i] (56:4)
  remark #25439: unrolled with remainder by 2
LOOP END
```

```
37 #ifdef NOALIAS
38 void matvec(int size1, int size2, FTYPE a[][size2], FTYPE b[restrict], FTYPE x[])
39 #else
40 void matvec(int size1, int size2, FTYPE a[][size2], FTYPE b[], FTYPE x[])
41 #endif
42 {
43     int i, j;
44
45     for (i = 0; i < size1; i++) {
46         b[i] = 0;
47
48 #ifdef ALIGNED
49 // The pragma vector aligned below tells the compiler to assume that the data in
50 // the loop is aligned on 16-byte boundary so the vectorizer can use
51 // aligned instructions to generate faster code.
52 #pragma vector aligned
53 #endif
54
55         for (j = 0; j < size2; j++) {
56             b[i] += a[i][j] * x[j];
57         }
58     }
59 }
```

Restrict is c99, not available in C++.
In these cases is compiler-specific,
e.g. `int *__restrict__ p`

Details

With -DNOALIAS

icc -std=c99 -O3 -qopt-report=5 -qopt-report-phase=vec -DNOALIAS -xHost Multiply.c Driver.c -o MatVector

See next slide for peeled and remainder concepts.
You already saw data alignment.

```
LOOP BEGIN at Multiply.c(55,3)
<Peeled loop for vectorization>
remark #15389: vectorization support: reference a[i][j] has unaligned access [ Multiply.c(56,12) ]
remark #15389: vectorization support: reference x[j] has unaligned access [ Multiply.c(56,22) ]
remark #15381: vectorization support: unaligned access used inside loop body
remark #15335: peel loop was not vectorized: vectorization possible but seems inefficient. Use vector a
remark #15305: vectorization support: vector length 8
remark #15309: vectorization support: normalized vectorization overhead 1.955
LOOP END
```

```
LOOP BEGIN at Multiply.c(55,3)
remark #15389: vectorization support: reference a[i][j] has unaligned access [ Multiply.c(56,12) ]
remark #15389: vectorization support: reference x[j] has unaligned access [ Multiply.c(56,22) ]
remark #15381: vectorization support: unaligned access used inside loop body
remark #15305: vectorization support: vector length 16
remark #15309: vectorization support: normalized vectorization overhead 2.733
remark #15300: LOOP WAS VECTORIZED
remark #15442: entire loop may be executed in remainder
remark #15450: unmasked unaligned unit stride loads: 2
remark #15475: --- begin vector cost summary ---
remark #15476: scalar cost: 8
remark #15477: vector cost: 0.930
remark #15478: estimated potential speedup: 6.460
remark #15488: --- end vector cost summary ---
LOOP END
```

```
LOOP BEGIN at Multiply.c(55,3)
<Remainder loop for vectorization>
remark #15389: vectorization support: reference a[i][j] has unaligned access [ Multiply.c(56,12) ]
remark #15389: vectorization support: reference x[j] has unaligned access [ Multiply.c(56,22) ]
remark #15381: vectorization support: unaligned access used inside loop body
remark #15305: vectorization support: vector length 8
remark #15309: vectorization support: normalized vectorization overhead 1.955
remark #15301: REMAINDER LOOP WAS VECTORIZED
LOOP END
LOOP END
```

```
LOOP BEGIN at Multiply.c(45,2)
remark #15344: loop was not vectorized: vector dependence prevents vectorization
remark #15346: vector dependence: assumed FLOW dependence between b[i] (46:3) and a[i][j] (56:4)
remark #15346: vector dependence: assumed ANTI dependence between a[i][j] (56:4) and b[i] (46:3)
```

```
LOOP BEGIN at Multiply.c(55,3)
remark #15344: loop was not vectorized: vector dependence prevents vectorization
remark #15346: vector dependence: assumed FLOW dependence between b[i] (56:4) and a[i][j] (56:4)
remark #15346: vector dependence: assumed ANTI dependence between a[i][j] (56:4) and b[i] (56:4)
remark #15346: vector dependence: assumed ANTI dependence between b[i] (56:4) and b[i] (56:4)
remark #15346: vector dependence: assumed FLOW dependence between b[i] (56:4) and b[i] (56:4)
remark #15346: vector dependence: assumed FLOW dependence between b[i] (56:4) and b[i] (56:4)
remark #15346: vector dependence: assumed ANTI dependence between b[i] (56:4) and b[i] (56:4)
LOOP END
```

Without -DNOALIAS

Other Transformations – Loop peeling

- Remove the first/s and/or the last/s iteration of the loop into separate code outside the loop
- It allows enforcing a particular initial memory alignment on array references prior to loop vectorization

LOOP BEGIN at gas_dyn2.f90(2330,26)

<Peeled>

remark #15389: vectorization support: reference AMAC1U has unaligned access

remark #15381: vectorization support: unaligned access used inside loop body

remark #15301: PEEL LOOP WAS VECTORIZED

LOOP END

LOOP BEGIN at gas_dyn2.f90(2330,26)

remark #25084: Preprocess Loopnests: Moving Out Store

remark #15388: vectorization support: reference AMAC1U has aligned access

remark #15399: vectorization support: unroll factor set to 2

remark #15300: LOOP WAS VECTORIZED

remark #15475: --- begin vector loop cost summary ---

remark #15476: scalar loop cost: 8

remark #15477: vector loop cost: 0.620

remark #15478: estimated potential speedup: 15.890

remark #15479: lightweight vector operations: 5

remark #15488: --- end vector loop cost summary ---

LOOP END

LOOP BEGIN at gas_dyn2.f90(2330,26)

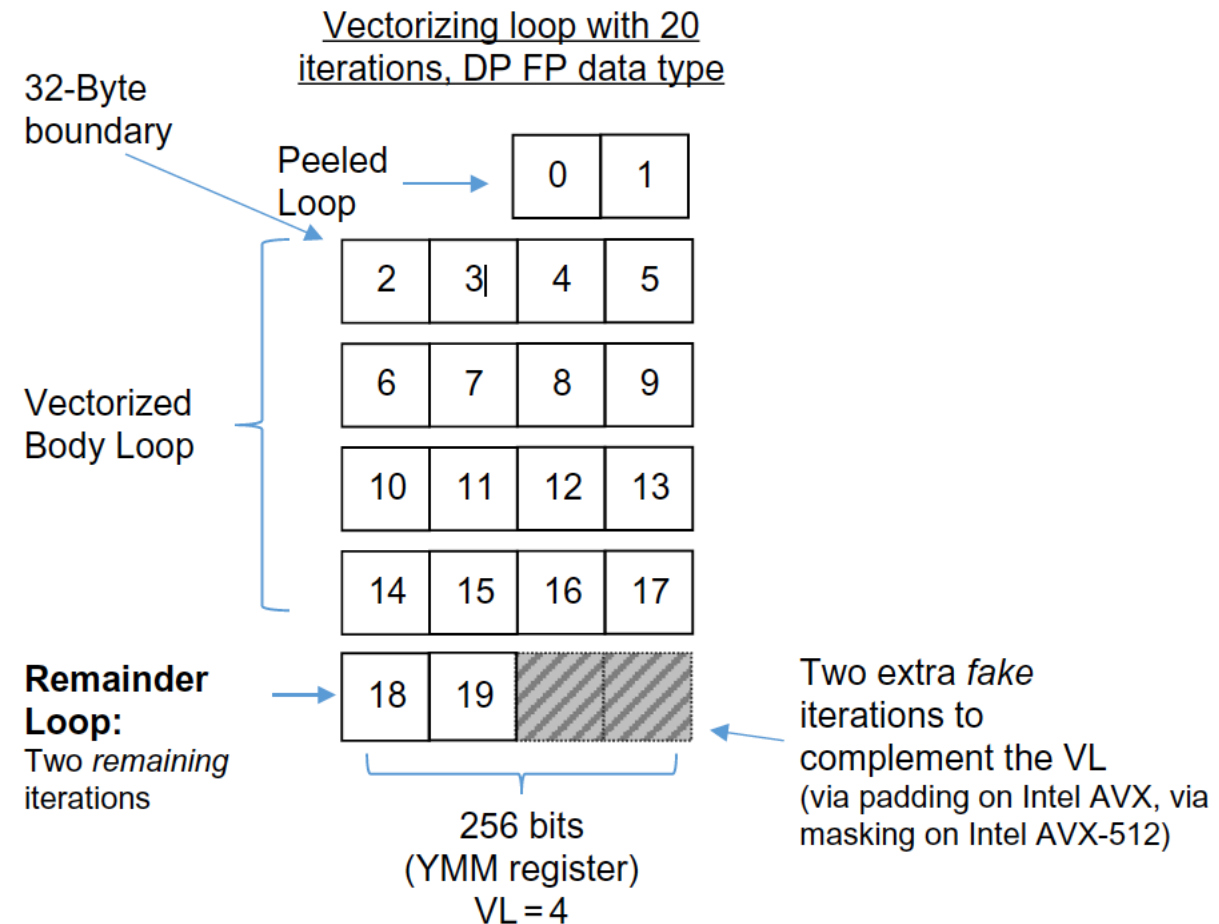
<Remainder>

remark #15388: vectorization support: reference AMAC1U has aligned access

remark #15388: vectorization support: reference AMAC1U has aligned access

remark #15301: REMAINDER LOOP WAS VECTORIZED

LOOP END



Remember

Xeon Phi has been designed for providing high performance **when using all the cores**.

It is however based on an Intel Atom architecture, therefore a classic Xeon processor like my E5645 provides the same or better performance for less \$ (about \$500 vs \$2,500).

Here the E5645 performance

- `icc -O0 -D NOFUNCCALL Multiply.c Driver.c -o MatVector`
Execution time is 49.071 seconds - GigaFlops = 0.415761
- `icc -O3 -D NOFUNCCALL Multiply.c Driver.c -o MatVector`
Execution time is 6.733 seconds - GigaFlops = 3.030179
- `icc -std=c99 -D NOALIAS -D ALIGNED -ipo Multiply.c Driver.c -o MatVector`
Execution time is **5.193** seconds - GigaFlops = **3.928563**
- `pgcc -fast -O3 -D NOALIAS -D ALIGNED Multiply.c Driver.c -o MatVector`
Execution time is 10.873 seconds - GigaFlops = 1.876454

Also linuxge.ge.infn.it is faster...

Another requirement for Vectorization: CORRECTNESS!

- Vectorization changes the order of computation compared to sequential case!
- Therefore, compiler must be able to prove that vectorization will produce correct result.
 - A first example was the pointer aliasing
- Need to consider independence of *unrolled* loop operations
- Compiler performs dependency analysis
- Rule of thumb: if you can execute the loop in the opposite order (e.g. for $i=n$; $i>0$; $i--$) it should be ok

Auto vectorization: not all loops will vectorize

Data dependencies between iterations

- Proven Read-after-Write data (i.e., loop carried) dependencies
- Assumed data dependencies
 - Aggressive optimizations (e.g., IPO) might help

Vectorization won't be efficient

- Compiler estimates how better the vectorized version will be
- Affected by data alignment, data layout, etc.

Unsupported loop structure

- While-loop, for-loop with unknown number of iterations
- Complex loops, unsupported data types, etc.
- (Some) function calls within loop bodies
 - Not the case for SVML functions

RaW dependency

```
for (int i = 0; i < N; i++)  
    a[i] = a[i-1] + b[i];
```

Inefficient vectorization

```
for (int i = 0; i < N; i++)  
    a[c[i]] = b[d[i]];
```

Function call within loop body

```
for (int i = 0; i < N; i++)  
    a[i] = foo(b[i]);
```

Loop-Carried Dependence vs. Loop Independent Dep.

They are determined by the relationships between statements in iterations of a loop

- LCD: when a statement in one iteration of a loop depends in some way on a statement in a **different iteration** of the same loop
- LID: if a statement in one iteration of a loop depends only on a statement **in the same iteration** of the loop

AND

- Two memory accesses are involved in a **data dependence** if they may refer to the same memory location **and** one of the references is a **write**

// Code block 1

```
for(i = 1; i < 4; i++)
```

```
  S1: b[i] = 8;
```

```
  S2: a[i] = b[i-1] + 10;
```

LCD

// Code block 2

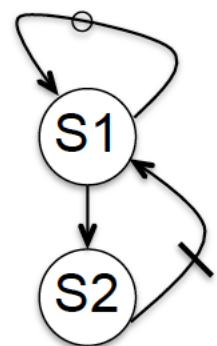
```
for(i = 0; i < 4; i++)
```

```
  S1: b[i] = 8;
```

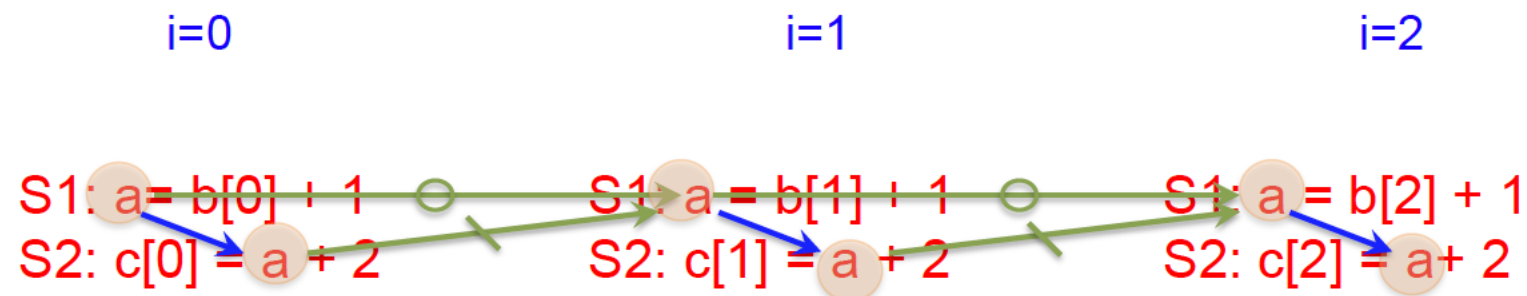
```
  S2: a[i] = b[i] + 10;
```

LID

LCD vs LID



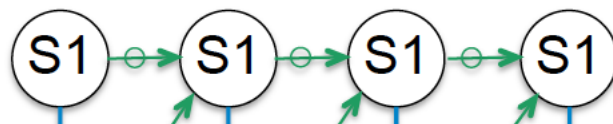
```
for (i=0; i<n; i++){  
  S1    a = b[i] + 1;  
  S2    c[i] = a + 2;  
}
```



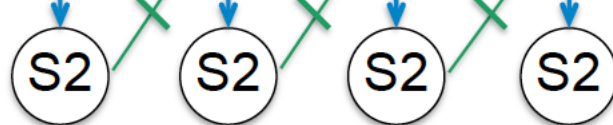
iteration:

0 1 2 3 ...

instances of S1:



instances of S2:



→ Loop independent dependence

→ Loop carried dependence

⊖ → Write after Write
+ → Write after read
| → Read after Write

Data Dependence

Read after Read is **not** a real dependency, thus always vectorizable

Read After Write

- Also called “flow” dependency
- Variable written first, then read
- Not vectorizable

```
for ( i=1; i<N; i++)  
    a[i] = a[i-1] + b[i];
```

S1	a = b + c;
S2	d = 2 * a;

Write after Read

- Also called “anti” dependency
- Variable read first, then written
- vectorizable

```
for ( i=0; i<N-1; i++)  
    a[i] = a[i+1] + b[i]
```

S1	c = a + b;
S2	a = 2 * a;

Write after Write

- a.k.a “output” dependency
- Variable written, then re-written
- Not vectorizable

```
for ( i=0; i<N; i++)  
    a[i%2] = b[i] + c[i];
```

S1	a = k; if (a>0) {
S2	a = 2 * c; }



Loop dependencies: RAW

Consider the loop:

$a = \{0, 1, 2, 3, 4\}$

$b = \{5, 6, 7, 8, 9\}$

```
for ( i=1; i<N; i++)  
    a[i] = a[i-1] + b[i];
```

Applying each operation sequentially:

$a[1] = a[0] + b[1] \rightarrow a[1] = 0 + 6 \rightarrow a[1] = 6$

$a[2] = a[1] + b[2] \rightarrow a[2] = 6 + 7 \rightarrow a[2] = 13$

$a[3] = a[2] + b[3] \rightarrow a[3] = 13 + 8 \rightarrow a[3] = 21$

$a[4] = a[3] + b[4] \rightarrow a[4] = 21 + 9 \rightarrow a[4] = 30$

$a = \{0, 6, 13, 21, 30\}$

Loop dependencies: RAW

Now let's try vector operations:

$a = \{0, 1, 2, 3, 4\}$

$b = \{5, 6, 7, 8, 9\}$

```
for ( i=1; i<N; i++)  
    a[i] = a[i-1] + b[i];
```

Applying vector operations, $i=\{1, 2, 3, 4\}$:

$a[i-1] = \{0, 1, 2, 3\}$ (load)

$b[i] = \{6, 7, 8, 9\}$ (load)

$\{0, 1, 2, 3\} + \{6, 7, 8, 9\} = \{6, 8, 10, 12\}$ (operate)

$a[i] = \{6, 8, 10, 12\}$ (store)

A should be $\{0, 6, 13, 21 \dots\}$

$a = \{0, 6, 8, 10, 12\} \neq \{0, 6, 13, 21, 30\}$ NOT VECTORIZABLE

Loop dependencies: WAR

Consider the loop:

$a = \{0, 1, 2, 3, 4\}$

$b = \{5, 6, 7, 8, 9\}$

```
for ( i=0; i<N; i++)  
    a[i] = a[i+1] + b[i];
```

Applying each operation sequentially:

$a[0] = a[1] + b[0] \rightarrow a[0] = 1 + 5 \rightarrow a[0] = 6$

$a[1] = a[2] + b[1] \rightarrow a[1] = 2 + 6 \rightarrow a[1] = 8$

$a[2] = a[3] + b[2] \rightarrow a[2] = 3 + 7 \rightarrow a[2] = 10$

$a[3] = a[4] + b[3] \rightarrow a[3] = 4 + 8 \rightarrow a[3] = 12$

$a = \{6, 8, 10, 12, 4\}$

Loop dependencies: WAR

Now let's try vector operations:

$a = \{0, 1, 2, 3, 4\}$

$b = \{5, 6, 7, 8, 9\}$

```
for ( i=0; i<N; i++)  
    a[i] = a[i+1] + b[i];
```

Applying vector operations, $i=\{1, 2, 3, 4\}$:

$a[i+1] = \{1, 2, 3, 4\}$ (load)

$b[i] = \{5, 6, 7, 8\}$ (load)

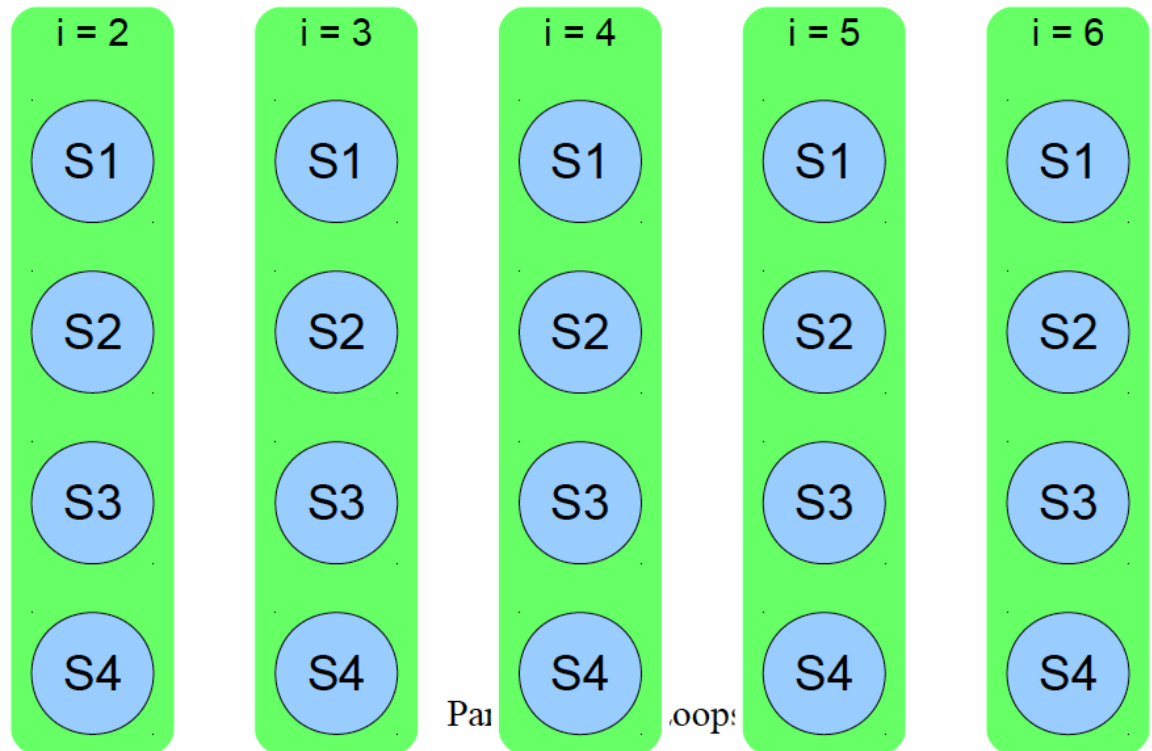
$\{1, 2, 3, 4\} + \{5, 6, 7, 8\} = \{6, 8, 10, 12\}$ (operate)

$a[i] = \{6, 8, 10, 12\}$ (store)

$a = \{0, 6, 8, 10, 12\} = \{0, 6, 8, 10, 12\}$ VECTORIZABLE

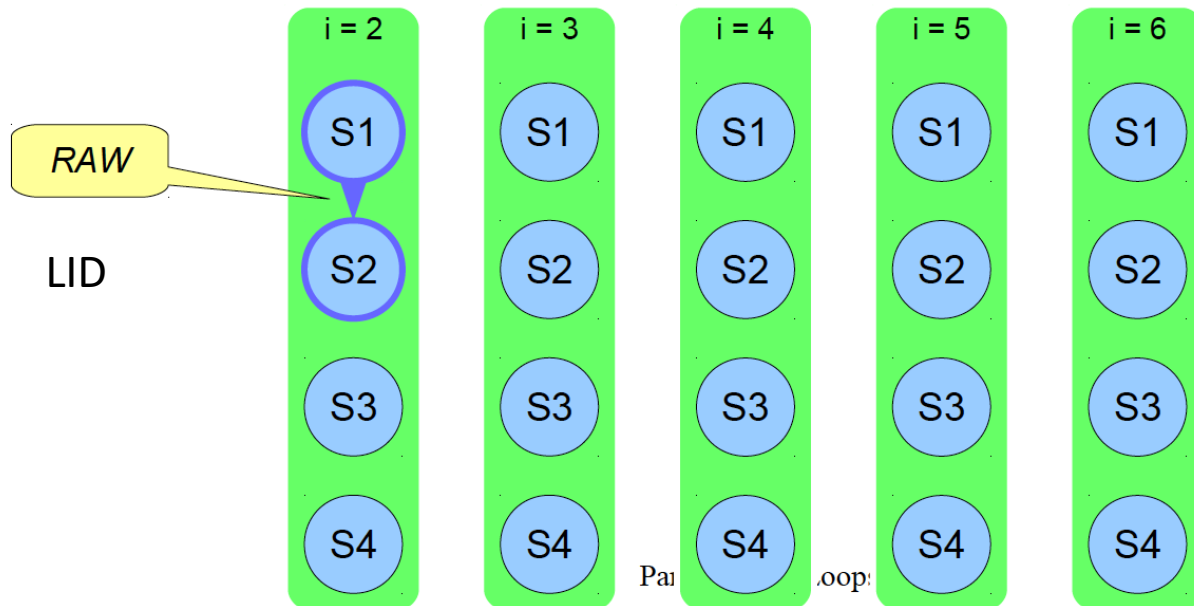
Exercise: draw
the dependencies

```
for (i=2; i<n; i++) {  
S1  a[i] = 4 * c[i-1] - 2;  
S2  b[i] = a[i] * 2;  
S3  c[i] = a[i-1] + 3;  
S4  d[i] = b[i] + c[i-2];  
}
```

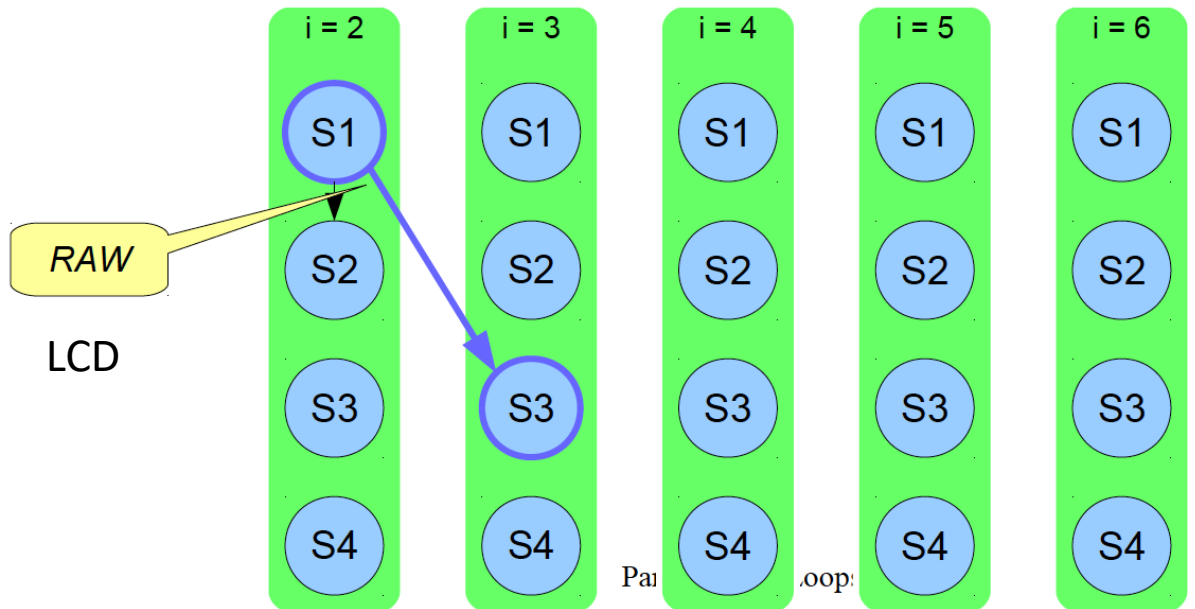


Dependencies on a[i]

```
for (i=2; i<n; i++) {  
S1 a[i] = 4 * c[i-1] - 2;  
S2 b[i] = a[i] * 2;  
S3 c[i] = a[i-1] + 3;  
S4 d[i] = b[i] + c[i-2];  
}
```



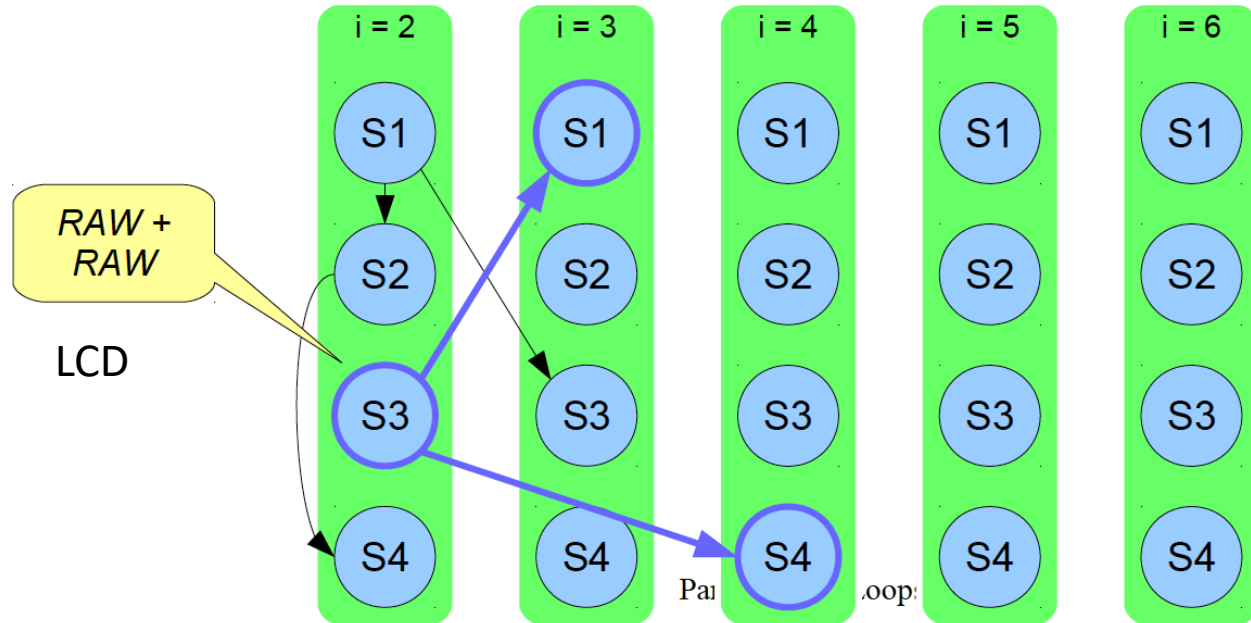
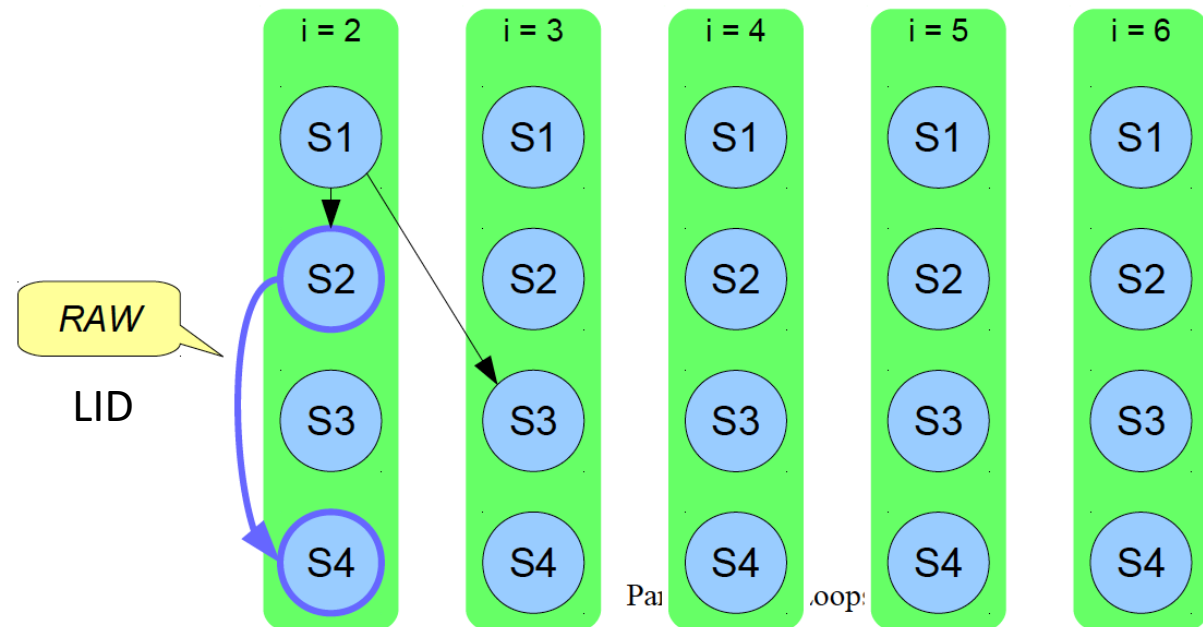
```
for (i=2; i<n; i++) {  
S1 a[i] = 4 * c[i-1] - 2;  
S2 b[i] = a[i] * 2;  
S3 c[i] = a[i-1] + 3;  
S4 d[i] = b[i] + c[i-2];  
}
```



Dependencies on b and c

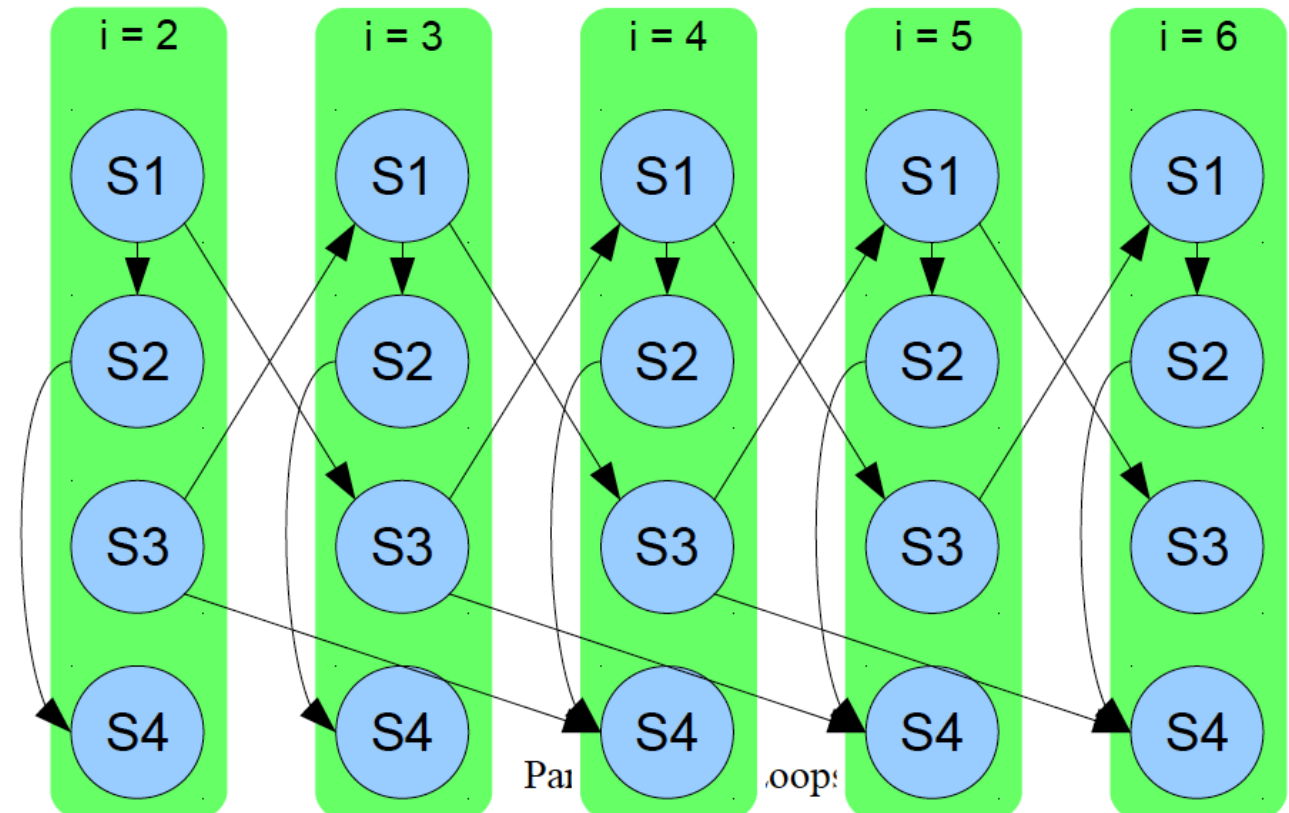
```
for (i=2; i<n; i++) {  
S1 a[i] = 4 * c[i-1] - 2;  
S2 b[i] = a[i] * 2;  
S3 c[i] = a[i-1] + 3;  
S4 d[i] = b[i] + c[i-2];  
}
```

```
for (i=2; i<n; i++) {  
S1 a[i] = 4 * c[i-1] - 2;  
S2 b[i] = a[i] * 2;  
S3 c[i] = a[i-1] + 3;  
S4 d[i] = b[i] + c[i-2];  
}
```



All the dependencies

```
for (i=2; i<n; i++) {  
S1  a[i] = 4 * c[i-1] - 2;  
S2  b[i] = a[i] * 2;  
S3  c[i] = a[i-1] + 3;  
S4  d[i] = b[i] + c[i-2];  
}
```



So?

Vectorization in most cases can be achieved by:

- Separating (distributing) the statements not in a cycle
- Removing dependences
- Freezing loops
- *Changing the algorithm*

In general, a statement inside a loop which is not in a cycle of the dependence graph (LCD) can be vectorized

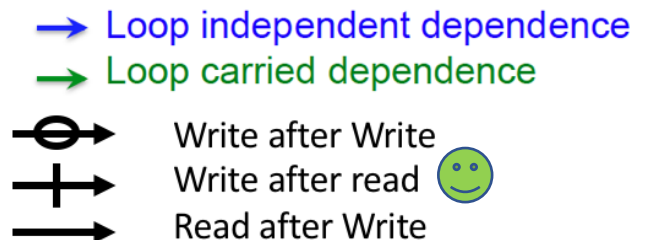
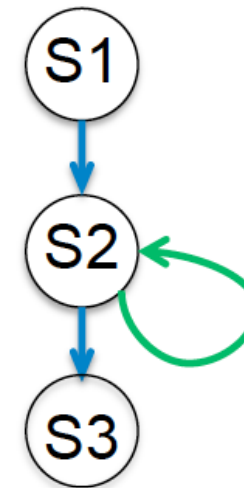
Distributing

Only S1 and S3 can be vectorized

```
for (i=2; i<n; i++){  
  S1 b[i] = b[i] + c[i];  
  S2 a[i] = a[i-1]*a[i-2]+b[i];  
  S3 c[i] = a[i] + 1;  
}
```



```
b[1:n-1] = b[1:n-1] + c[1:n-1];  
for (i=1; i<n; i++){  
  a[i] = a[i-1]*a[i-2]+b[i];  
}  
c[1:n-1] = a[1:n-1] + 1;
```



It means vectorized

Removing dependences

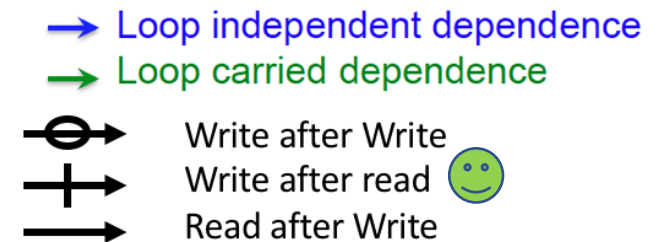
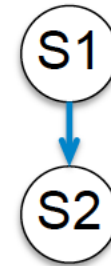
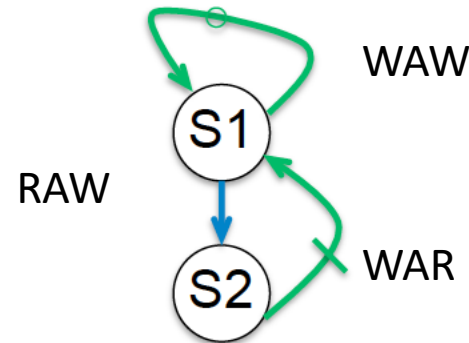
```
for (i=0; i<n; i++){  
S1   a = b[i] + 1;  
S2   c[i] = a + 2;  
}
```



```
for (i=0; i<n; i++){  
S1   a'[i] = b[i] + 1;  
S2   c[i] = a'[i] + 2;  
}  
a=a'[n-1]
```



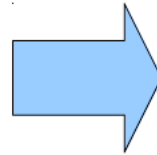
```
S1   a'[0:n-1] = b[0:n-1] + 1;  
S2   c[0:n-1] = a'[0:n-1] + 2;  
      a=a'[n-1]
```



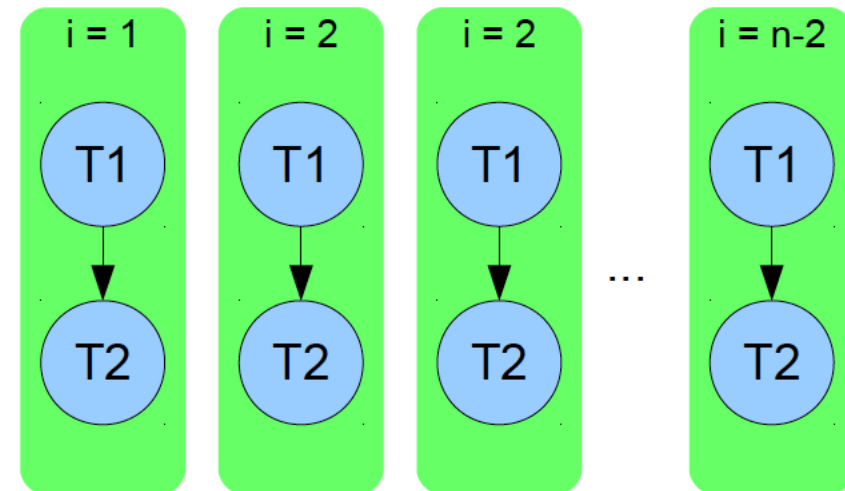
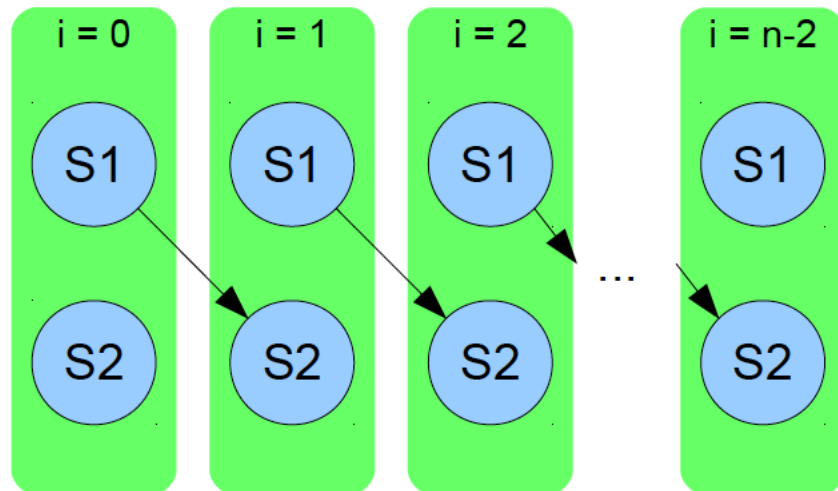
Removing dependencies: loop aligning

- Dependencies can sometimes be removed by **aligning** loop iterations

```
a[0] = 0;  
for (i=0; i<n-1; i++) {  
  S1 a[i+1] = b[i] * c[i];  
  S2 d[i] = a[i] + 2;  
}
```

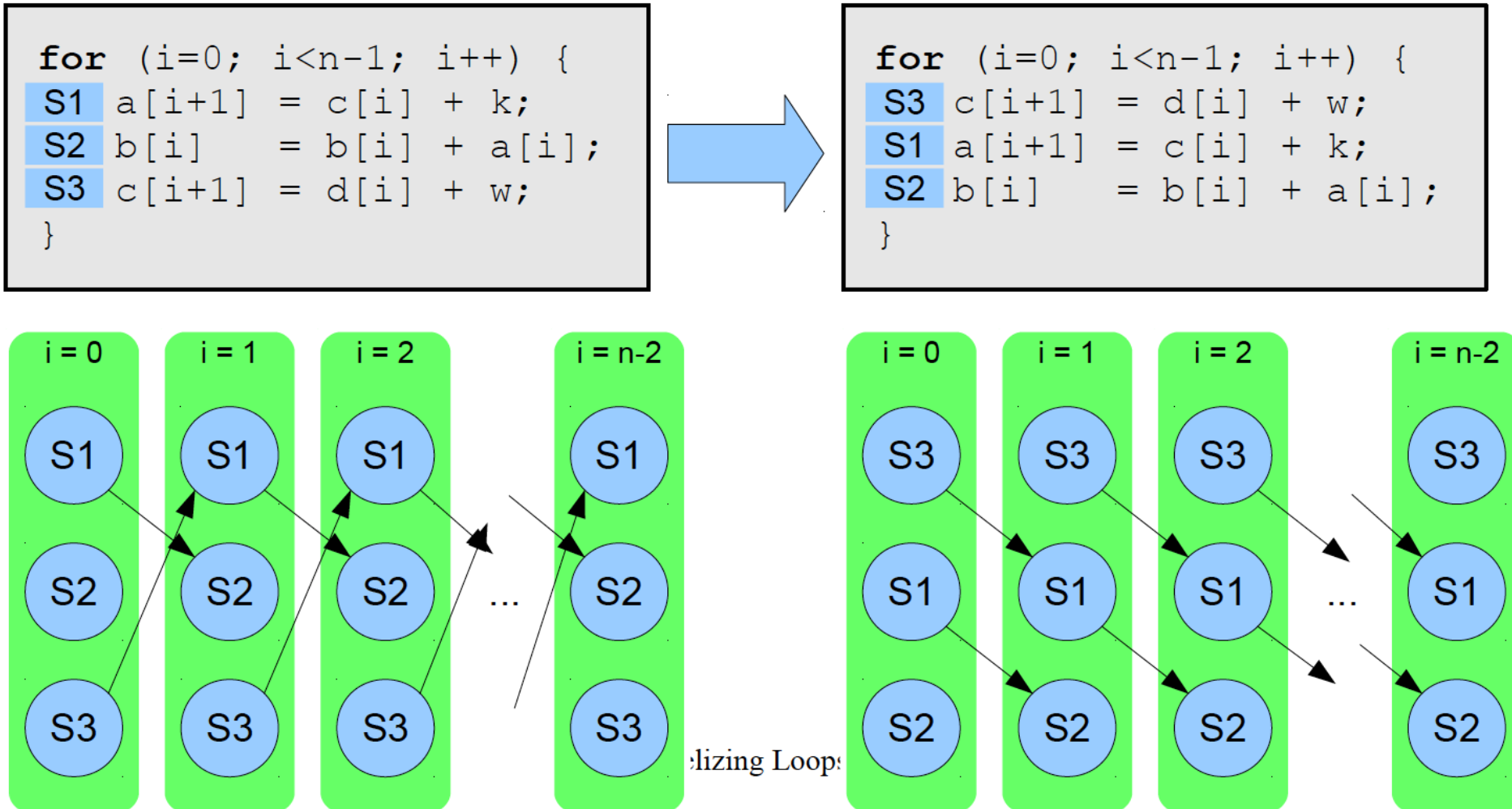


```
a[0] = 0;  
d[0] = a[0] + 2;  
for (i=1; i<n-1; i++) {  
  T1 a[i] = b[i-1] * c[i-1];  
  T2 d[i] = a[i] + 2;  
}  
a[n-1] = b[n-2] * c[n-2];
```



Removing dependencies: Reordering 1

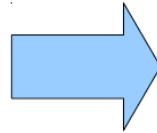
- Some dependencies can be removed by **reordering**



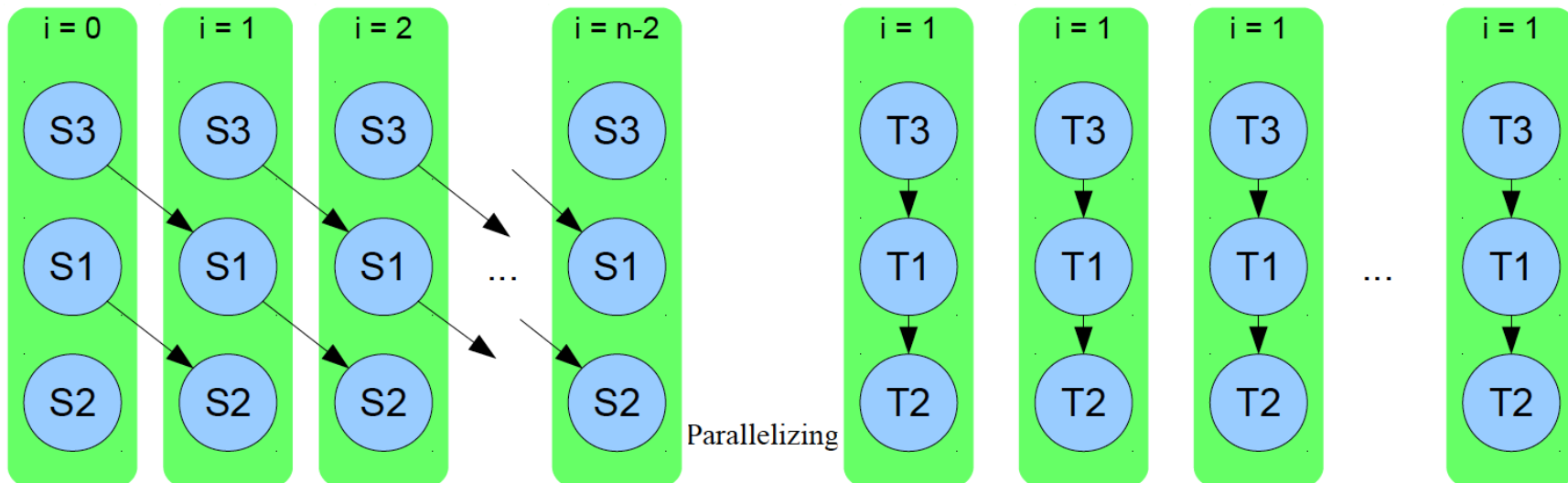
Removing dependencies: Reordering 2

- After reordering, we can align loop iterations

```
for (i=0; i<n-1; i++) {  
  S3 c[i+1] = d[i] + w;  
  S1 a[i+1] = c[i] + k;  
  S2 b[i] = b[i] + a[i];  
}
```

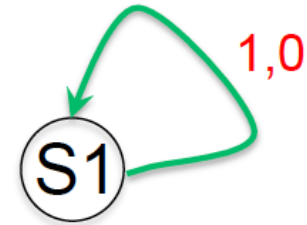


```
b[0] = b[0] + a[0];  
a[1] = c[0] + k;  
b[1] = c[1] + a[1];  
for (i=1; i<n-2; i++) {  
  T3 c[i] = d[i-1] + w;  
  T1 a[i+1] = c[i] + k;  
  T2 b[i+1] = b[i+1] + a[i+1];  
}  
c[n-2] = d[n-3] + w;  
a[n-1] = c[n-2] + k;  
c[n-1] = d[n-2] + w;
```



Freezing Loops

```
for (i=1; i<n; i++) {  
    for (j=1; j<n; j++) {  
        a[i][j]=a[i][j]+a[i-1][j];  
    }  
}
```



Ignoring (freezing) the outer loop:

```
for (j=1; j<n; j++) {  
    a[i][j]=a[i][j]+a[i-1][j];  
}
```



```
for (i=1; i<n; i++) {  
    a[i][1:n-1]=a[i][1:n-1]+a[i-1][1:n-1];  
}
```

Reduction

- A reduction variable (or simply "reduction") is an operation that happens in a specific variable, in every iteration of a loop.
- It is in the form: `new_value = old_value OP data`.
- That is, in every iteration, the new value of the variable is an operation involving the old value and some data.
 - a reduction is a special kind of a loop-carried dependency
 - But it can be parallelised for associative Ops: $((1 + 2) + 3) + 4 == (1+2) + (3+4), (3+1)+(2+4)...$
 - The important thing here is that the OP is the *same* in *every* iteration.

```
int sum_of_array(int *a, int len) {  
    int sum = 0;  
    for (int i = 0; i < len; ++i) sum = sum + a[i];  
}
```

#pragma simd for icc

```
void add_floats(float *a, float *b, float *c, float *d, float *e, int n){  
    int i;  
    #pragma simd  
    for (i=0; i<n; i++) a[i] = a[i] + b[i] + c[i] + d[i] + e[i];  
}
```

In the example above, the function `add_floats()`

- uses too many unknown pointers for the compiler's automatic runtime independence check optimization
- has a reduction

The programmer can enforce the vectorization of this loop by using the *simd* pragma to avoid the overhead of runtime check.

Non-Intel compiler can exploit `#pragma omp simd`

<https://www.intel.com/content/www/us/en/develop/documentation/cpp-compiler-developer-guide-and-reference/top/compiler-reference/pragmas/intel-specific-pragma-reference/simd.html>

Further Hinting and Pragmas

```
void vec1(double s1, int M, int N, double *x) {  
    ...  
    for (i=ib; i<ie; i++) { x[i] = x[i+M] + s1;  
    }
```

- In this case, the Intel compiler is able to recognize that certain **negative** values of M will result in a read-after-write dependency.
- But the compiler is not entirely prevented from vectorizing the loop; instead, it produces **multiversi**oned code that can branch to either a vectorized or an unvectorized loop, depending on the value of M!
- In many ways this outcome is better than if we force the compiler to vectorize the loop through the compiler directives presented below. Why take the risk of generating incorrect results through the heavy-handed use of directives?

Further Hinting and Pragmas

- There are other instances where the compiler cannot so easily distinguish between safe and unsafe cases.
- In such instances, compiler hints via flags or pragmas can be the most suitable way to ensure
- `#pragma ivdep` (Ignore Vector DEpendencies)
It implies (or guarantees) that any apparent data dependencies are safe to ignore.
- NOTE: if the programmer is wrong, and the loop actually does have a "bad" dependency like read after write (as previous code with $M < 0$), the program will likely produce **incorrect** results

Further Hinting and Pragmas

```
void foo(int lb, int ub) {  
    float a = 0;  
    #pragma omp simd early_exit reduction(+:a)  
    for(j=lb; j<ub; j++) {  
        if (b[j] <= 0 )  
            break;  
        a += b[j];  
    }  
}
```

#pragma omp simd early_exit

- icc specific
- Defines that the vector execution of the *for* loop is safe even though the loop may exit before the loop upper bound condition $j < ub$ becomes false
- Safety of such vector evaluation is programmer's responsibility.

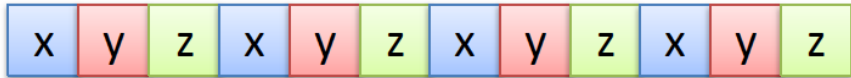
Recap on data access

- Data travel back and forth between memory levels in units of a cache line, about 64 bytes (8 doubles).
- Data properly aligned (i.e. allocated) move readily from registers to cache to memory in single operations.
- In the case of arguments to functions or subroutines, though, the compiler may not know for certain that all the variables given as arguments will be suitably aligned. You should specify it.
- It should be clear enough at this point of the course how to properly access both one- and multi-dimensional arrays.
- But what about structs?

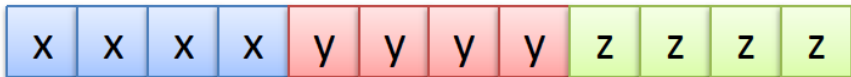
Data layout

Use Structure of Arrays (SoA)
instead of Array of Structures (AoS).

```
struct Vector3s { //AoS
    double x;
    double y;
    double z;
};
```



```
struct Vectors3s { //SoA
    double* x;
    double* y;
    double* z;
};
```



Array of Structures vs Structure of Arrays

```
// Array of Structures (AoS)
struct coordinate {
    float x, y, z;
} crd[N];

...
for (int i = 0; i < N; i++)
    ... = ... f(crd[i].x, crd[i].y, crd[i].z);
```

Consecutive elements in memory →

x0 y0 z0 x1 y1 z1 ... x(n-1) y(n-1) z(n-1)

```
// Structure of Arrays (SoA)
struct coordinate {
    float x[N], y[N], z[N];
} crd;

...
for (int i = 0; i < N; i++)
    ... = ... f(crd.x[i], crd.y[i], crd.z[i]);
```

Consecutive elements in memory →

x0 x1 ... x(n-1) y0 y1 ... y(n-1) z0 z1 ... z(n-1)

Example

```
struct Person
{
    Person(const string& n, uint8_t a, uint32_t d)
        : name(n), age(a), dob(d) {}

    string name;
    uint8_t age;
    uint32_t dob;
};

using VP = vector<Person>;

void addPerson(VP& v, Person&& p) { v.push_back(move(p)); }

uint64_t averageNameLen(const VP& v)
{
    return accumulate(begin(v), end(v), (uint64_t)0,
        [](auto sum, auto& p) { return sum + p.name.length(); }) / v.size();
}
```

```
struct Persons
{
    vector<string> names;
    vector<uint8_t> ages;
    vector<uint32_t> dobs;

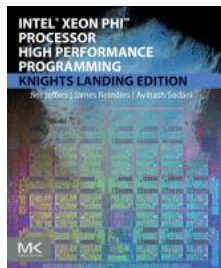
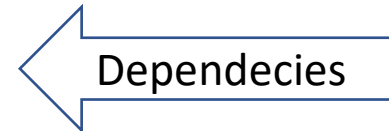
    void addPerson(const string& n, uint8_t a, uint32_t d)
    {
        names.push_back(n);
        ages.push_back(a);
        dobs.push_back(d);
    }

    uint64_t averageNameLen() const
    {
        return accumulate(begin(names), end(names), (uint64_t)0,
            [](auto sum, auto& n) { return sum + n.length(); }) / names.size();
    }
}
```

Try `icpc -std=c++17 array-structure.cpp -o array-structure` (99 vs 62)
and `icpc -std=c++17 -xHost array-structure.cpp -o array-structure` (92 vs 28)

Useful links

- <http://www.sci.utah.edu/~mb/Teaching/Week5/9-Vectorization.pdf>
- <https://software.intel.com/en-us/advisor/documentation/get-started>
- <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference-automatic-vectorization>
- <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference-vectorization-and-loops>
- [https://hpc-forge.cineca.it/files/ScuolaCalcoloParallelo_WebDAV/public/anno-2017/26th Summer School on Parallel Computing/Bologna/SCP-KNL-vectorization.pdf](https://hpc-forge.cineca.it/files/ScuolaCalcoloParallelo_WebDAV/public/anno-2017/26th_Summer_School_on_Parallel_Computing/Bologna/SCP-KNL-vectorization.pdf)
- <https://www.sciencedirect.com/science/article/pii/B9780128091944000090>



Exercise 1

Compile the simple_dp.c and simple_sp.c codes (Aulaweb – Vect Examples) with the following combinations

- `icc -qopenmp -no-vec simple_XX.c`
- `icc -qopenmp simple_XX.c`
- `icc -qopenmp -xHost simple_XX.c`
- What are the timings?
- Do you notice anything about the value of sum with non-vectorized and vectorized codes ?
- And what happens if you add, after the main loop?

```
for(i=1;i<VECSIZE;i++) {  
    a[i]=a[i]+a[i-1];  
}
```

Solution

	Non vect	Vect	AVX-512	Sum non-vect	Sum vect
SP	4.641423	0.145412	0.036836	1119.203491210938	1100.000488281250
DP	4.643434	0.290392	0.073117	1100.000000058394	1100.000000000005

1) $T(SP) = T(DP)/2$

2) Vectorized code might give slightly different results because some non-vectorized instructions act on different representation (e.g. 80-bit floating point internally instead of 64-bit for double precision)

3) LOOP BEGIN at simple2_sp.c(27,3)

remark #15344: loop was not vectorized: vector dependence prevents vectorization

remark #15346: vector dependence: assumed FLOW dependence between a[i] (28:5) and a[i-1] (28:5)

LOOP END

Floating point sum

- You must move the decimal point of the number that has the smaller exponent so as to align it with the larger one.
- Let us consider, for example, $9,999 \cdot 10^1 + 1,610 \cdot 10^{-1}$
- Step 1: alignment
 $0,01610 \cdot 10^1 + 9,999 \cdot 10^1$
- Step 2: sum
 $10,0151 \cdot 10^1$
- Step 3: conversion to the scientific normalised form
 $1,00151 \cdot 10^2$
- BUT what happens if I have only 4 digits?
In step 2 i will use $0,016 \cdot 10^1$, tso the result will be $1,002 \cdot 10^2$

Problems

La limitatezza della precisione dell'aritmetica dei calcolatori porta ad avere dei problemi con le proprietà delle operazioni aritmetiche.

Ad esempio, **la somma in virgola mobile non è associativa**, cioè, in generale, **non è vero che $x+(y+z) = (x+y)+z$**

I problemi nascono quando si vogliono sommare due numeri molto grandi di segno opposto, con altro un numero molto piccolo

$$\begin{aligned}x &= 1.5_{10} \cdot 10^{38} \\ y &= -1.5_{10} \cdot 10^{38} \\ z &= 1.0_{10}\end{aligned} \quad \text{con } x,y,z \text{ espressi in singola precisione}$$

$$\begin{aligned}x+(y+z) &= -1.5 \cdot 10^{38} + (1.5 \cdot 10^{38} + 1.0) \\ &= -1.5 \cdot 10^{38} + 1.5 \cdot 10^{38} = 0.0_{10}\end{aligned}$$

Dovrei spostare l'1 di 38 posizioni a destra

$$\begin{aligned}(x+y)+z &= (-1.5 \cdot 10^{38} + 1.5 \cdot 10^{38}) + 1.0 \\ &= 0.0 + 1.0 = 1.0_{10}\end{aligned}$$

Arithmetic accuracy

- L'hardware utilizza bit aggiuntivi per i calcoli, onde poter arrotondare in modo più accurato
- Lo standard prevede due bit aggiuntivi a destra della mantissa, la guardia e l'arrotondamento
 - Provare cosa succede sommando $2,56$ con $2,34 \cdot 10^2$ con o senza due cifre aggiuntive
- Vedrete questi aspetti in un corso successivo

Exercise 2 and 3

- `icc -xHost -qopenmp alias.c`
Addition took 0.072852 seconds
- `icc -xHost -qopenmp alias_func.c alias_main.c`
Addition took 0.276665 seconds
- Try to solve the issues with `-guide`
- Analyze and try to speedup `forward.c` by removing the dependencies

Solutions

- Exercise 2: it is sufficient to compile with -fargument-noalias
But nothing changes...

Try with -no-vec to see the expected non vectorized execution time and -qopt-report=5 -qopt-report-phase=vec to see that the compiler produced two alternative versions of the loop

- One for the case update_final(a,b,VECSIZE,ITERATIONS);
- The second for update_final(b,b,VECSIZE,ITERATIONS);

```
LOOP BEGIN at alias_func.c(9,5)
<Multiversed v2>
  remark #15304: loop was not vectorized: non-vectorizable loop instance from multiversioning
LOOP END
```

- Exercise 3: split the loop but pay attention to the final sum!

To summarize

1. use a compiler switch for auto-vectorization (and hope it vectorizes)
2. interact with the compiler to get hints
-*qopt-report=5 -qopt-report-phase=vec*
and give it your compiler hints (and hope it vectorizes)
3. code explicitly to vectorize the loop
4. disregard vectorization and use explicit parallelism

C99 *restrict* keyword IVDEP (ignore assumed vector dependencies)

```
void v_add (float *restrict c,  
            float *restrict a,  
            float *restrict b)  
{  
    for (int i=0; i<= MAX; i++)  
        c[i]=a[i]+b[i];  
}
```

```
void v_add (float *c,  
            float *a,  
            float *b)  
{  
    #pragma ivdep  
    for (int i=0; i<= MAX; i++)  
        c[i]=a[i]+b[i];  
}
```

```
void v_add (float *c,  
            float *a,  
            float *b)  
{  
    #pragma omp simd  
    for (int i=0; i<= MAX; i++)  
        c[i]=a[i]+b[i];  
}
```

Profiling and Tuning

- Actually, the first step for improving the performance of a program is the profiling, to understand where are the hotspots
- Many Tools
 - Program code
 - GNU gprof
 - Nvidia Nsight <https://developer.nvidia.com/nsight-systems>
 - Intel VTUNE <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html>
- Vtune tutorials
<https://www.intel.com/content/www/us/en/develop/documentation/vtune-help/top/introduction/tutorials-and-samples.html>
- From the profiling to support for the optimization of the code
 - Intel Advisor
<https://www.intel.com/content/www/us/en/developer/tools/oneapi/advisor.html>

Intel® VTune™ Profiler

Locate performance bottlenecks fast. Advanced sampling and profiling techniques quickly analyze your code, isolate issues, and deliver insights for optimizing performance on modern processors.



Single Thread

Optimize single-threaded performance.



Multithread

Effectively use all available cores.



System

See a system-level view of application performance.



Media & OpenCL™ Applications

Deliver high-performance image and video processing pipelines.



HPC & Cloud

Access specialized, in-depth analyses for HPC and cloud computing.



Memory & Storage Management

Diagnose memory, storage, and data plane bottlenecks.



Analyze & Filter Data

Mine data for answers.

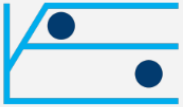


Environment

Fits your environment and workflow.

Intel® Advisor

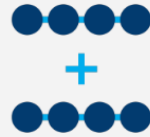
Design and optimize high-performing code for modern computer architectures. Effectively use more cores, vectorization, memory, and heterogeneous processing.



Roofline Analysis

Optimize your application for memory and compute.

[Read More](#) →



Vectorization Optimization

Enable more vector parallelism and improve its efficiency.

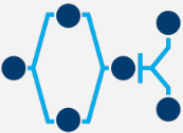
[Read More](#) →



Thread Prototyping

Model, tune, and test multiple threading designs.

[Read More](#) →



Build Heterogeneous Algorithms

Create and analyze data flow and dependency computation graphs.

Timing with program code

For benchmarking, an accurate wallclock timer (end-to-end stop watch) is required:

- `clock_gettime()`, POSIX compliant timing function
- `MPI_Wtime()` and `omp_get_wtime()`, standardized programming-model-specific timing routines for MPI and OpenMP

```
#include <stdlib.h>
#include <time.h>

double getTimeStamp()
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1.e-9;
}
```

Usage:

```
double S, E;
S = getTimeStamp();
/* measured code region */
E = getTimeStamp();
return E-S;
```

Sampling-based runtime profile with perf

Instrumentation based with gprof

Compile with `-pg` switch:

```
icc -pg -O3      myfile1.c
```

Execute the application. During execution a file `gmon.out` is generated.

Analyze the results with:

```
gprof ./a.out | less
```

The output contains three parts: A flat profile, the call graph, and an alphabetical index of routines.

The flat profile is what you are usually interested in.

Example

Each sample counts as 0.01 seconds.

Time spent in routine itself		How often was it called		How much time was spent per call		
%	cumulative	self	calls	self	total	name
time	seconds	seconds		s/call	s/call	
66.86	26.14	26.14	502	0.05	0.05	ForceLJ::compute(Atom&, Neighbor&, Comm&, int)
30.77	38.17	12.03	26	0.46	0.46	Neighbor::build(Atom&)
1.43	38.73	0.56	1	0.56	38.46	Integrate::run(Atom&, Force*, Neighbor&, Comm&, Thermo&, Timer&)
0.36	38.87	0.14	2850	0.00	0.00	Atom::pack_comm(int, int*, double*, int*)
0.15	38.93	0.06	2850	0.00	0.00	Atom::unpack_comm(int, int, double*)
0.13	38.98	0.05	26	0.00	0.00	Atom::pbc()
0.10	39.02	0.04				__intel_ssse3_rep_memcpy
0.08	39.05	0.03	25	0.00	0.00	Atom::sort(Neighbor&)
0.08	39.08	0.03	1	0.03	0.03	create_atoms(Atom&, int, int, int, double)
0.05	39.10	0.02	26	0.00	0.00	Comm::borders(Atom&)
0.00	39.10	0.00	1221559	0.00	0.00	Atom::pack_border(int, double*, int*)
0.00	39.10	0.00	1221559	0.00	0.00	Atom::unpack_border(int, double*)
0.00	39.10	0.00	131072	0.00	0.00	Atom::addatom(double, double, double, double, double, double)
0.00	39.10	0.00	1025	0.00	0.00	Timer::stamp(int)
0.00	39.10	0.00	502	0.00	0.00	Thermo::compute(int, Atom&, Neighbor&, Force*, Timer&, Comm&)
0.00	39.10	0.00	500	0.00	0.00	Timer::stamp()
0.00	39.10	0.00	475	0.00	0.00	Comm::communicate(Atom&)
0.00	39.10	0.00	26	0.00	0.00	Comm::exchange(Atom&)
0.00	39.10	0.00	25	0.00	0.00	Timer::stamp_extra_stop(int)
0.00	39.10	0.00	25	0.00	0.00	Timer::stamp_extra_start()
0.00	39.10	0.00	25	0.00	0.00	Neighbor::binatoms(Atom&, int)
0.00	39.10	0.00	7	0.00	0.00	Timer::barrier_stop(int)
0.00	39.10	0.00	1	0.00	0.00	create_box(Atom&, int, int, int, double)
0.00	39.10	0.00	1	0.00	0.00	create_velocity(double, Atom&, Thermo&)

Intel Advisor

- <https://software.intel.com/en-us/advisor-tutorial-roofline-roofline-use-case>

- `icc -g -O2 -std=c++11 roofline.cpp`

- For the analysis here
add `-qopenmp`

and select

`#define GROUP_4`

`#define G$ _SOA`

```
/******  
// #define GROUP_1  
#ifdef GROUP_1  
// #define G1_AOS_SCALAR  
#define G1_SOA_SCALAR  
// #define G1_SOA_VECTOR  
#endif /******  
// #define GROUP_2  
#ifdef GROUP_2  
#define G2_AOS_SCALAR  
#define G2_SOA_SCALAR  
#define G2_SOA_VECTOR  
#endif /******  
// #define GROUP_3  
#ifdef GROUP_3  
#define G3_AOS_SCALAR  
// #define G3_SOA_SCALAR  
// #define G3_AOS_VECTOR  
// #define G3_SOA_VECTOR  
// #define G3_SOA_VECTOR_FMAS  
#endif /******  
#define GROUP_4  
#ifdef GROUP_4  
#define G4_SOA  
// #define G4_AOSOA  
#endif /******  
  
#define MAXVALUE 1000000  
#define ARRAY_SIZE_1 1328  
#define REPEAT_1 10000000  
#define ARRAY_SIZE_2 20000  
#define REPEAT_2 30000000  
#define UNROLL_COUNT 2
```

Group 1: $X = Y_a + Y_b$

Group 2: $X = Y_a + Y_b + Y_b$

Group 3: $X = Y_a + Y_a + Y_b + Y_b + Y_b$

Group 4: $X = Y_a + Y_a + Y_b + Y_b$

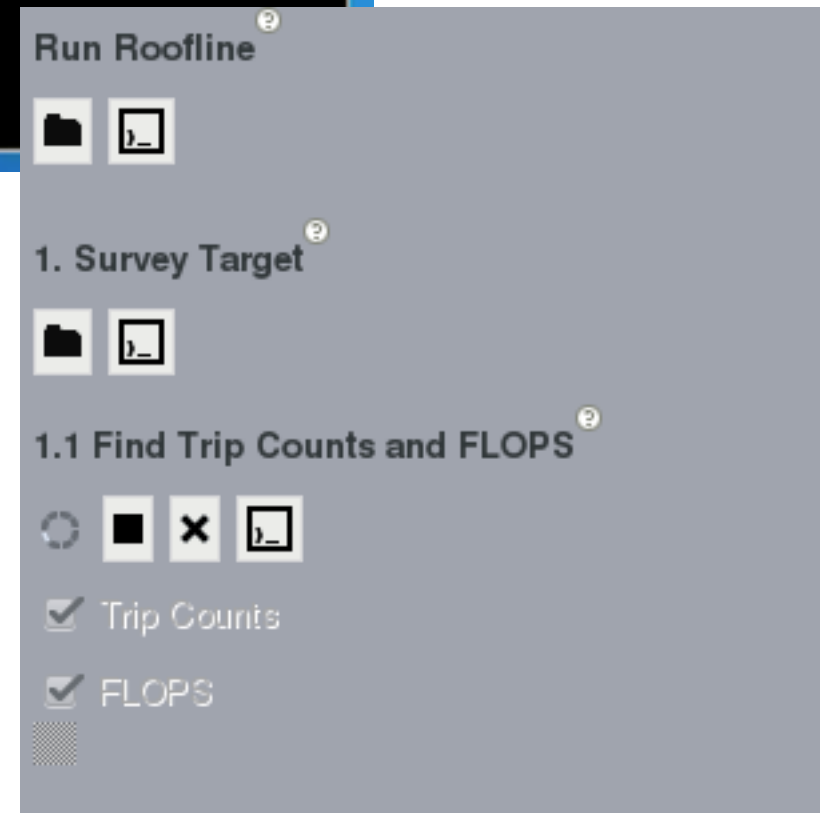
Analysis

```
name@machine:~$ advixe-cl --collect survey --project-dir ./outputfolder -- yourprogram
name@machine:~$ advixe-cl --collect tripcounts -flops-and-masks --project-dir ./outputfolder -- yourprogram
```

Command line

GUI

You can recompile your code many times and repeat the analyses keeping the name of the executable



In details... roofline

Load at least these scripts

```
./opt/intel/parallel_studio_xe_2019/bin/psxevars.sh intel64
```

```
./opt/intel/parallel_studio_xe_2019/advisor_2019/advixe-vars.sh
```

```
advixe-cl -collect survey -- executable
```

```
advixe-cl -report survey -format=xml -- executable (or csv, text)
```

```
advixe-cl -collect tripcounts -flops -- executable
```

```
advixe-cl -collect roofline --project-dir=. -- executable
```

```
advixe-cl -report roofline -project-dir=. -report-output=./filename.html
```

```
advixe-cl --help collect
```

```
advixe-cl --help report
```

<https://software.intel.com/content/www/us/en/develop/articles/tutorial-intel-advisor-vectorization-sample-readme.html>

In depth analysis

`advixe-cl -collect survey -- executable`

`advixe-cl -collect tripcounts -flop -- executable`

`advixe-cl -collect dependencies -- executable`

`advixe-cl -collect map -- executable`

Then analyse on your local PC

`scp e000/ Vectorization_Advisor.advixeproj ...`



Higher instruction set architecture (ISA) available

Consider recompiling your application using a higher ISA.

SURVEY

Performance (GFLOPS)

Use Single-Threaded Roofs



Self Elapsed Time: 94.115 s Total Time: 94.115 s

Arithmetic Intensity (FLOP/Byte)

File: roofline.cpp:310 main

Line	Source	Total Time	%	Loop/Function Time	%	Traits
304	for (int r = 0; r < REPEAT_2; r++)					
305	{					
306	for (int j = 0; j < 2; j++)					
307	{					
308	#pragma nounroll					
309	#pragma omp simd simdlen(VECTOR_LENGTH)					
310	for (int i = 0; i < ARRAY_SIZE_2 / 2; i++)	24,000s		94,115s		
311	{					
312	AoSoA_X[(j * (ARRAY_SIZE_2 / 2)) + i] = AoSoA_Y[j].a[i] + AoSoA_Y[j].a[i]	35,480s				
313	+ AoSoA_Y[j].b[i] + AoSoA_Y[j].b[i];	34,635s				
314	}					
315	}					
Selected (Total Time):		24,000s				



Program metrics

Elapsed Time 94,61s

Vector Instruction Set SSE, SSE2

Total GFLOP Count 180,00

Total Arithmetic Intensity ⓘ 0,12

Number of CPU Threads 1

Total GFLOPS 1,90



Loop metrics

Metrics

Total

Total CPU time

94,60s



100,0%

Time in 1 vectorized loop

94,12s



99,5%

Time in scalar code

0,48s



Vectorization Gain/Efficiency

Vectorized Loops Gain/Efficiency ⓘ 2,67x



67%

ⓘ

FileViewHelp

Welcomee000

Vectorization Workflow

Threading Workflow

OFFBatch mode

Run Roofline

1. Survey Target

1.1 Find Trip Counts and FLOPS

2.1 Check Dependencies

2.2 Check Memory Access Patterns

Re-finalize Survey

Elapsed time: 94,29s

VectorizedNot Vectorized

FILTER: All ModulesAll Sources

SummarySurvey & RooflineRefinement Reports

Site Location

Loop-Carried Dependencies

Strides Distribution

Access Pattern

Max. Site Footprint

Site Name

Recommendations

[loop in main at roofline.cpp:...

No information available

100% / 0% / 0%

All unit strides

240B

loop_site_4

308#pragma nounroll

309#pragma omp simd simdlen(VECTOR_LENGTH)

310for (int i = 0; i < ARRAY_SIZE_2 / 2; i++)

311{

312AoSoA_X[(j * (ARRAY_SIZE_2 / 2)) + i] = AoSoA_Y[j].a[i] + AoSoA_Y[j].a[i]

Memory Access Patterns Report

Dependencies Report

Recommendations

ID

Stride

Type

Source

Nested Function

Variable references

Access Footprint

Modules

Site Name

Access Type

+P1

1

Unit stride

roofline.cpp:312

AoSoA_Y

240B

libc.so.6; roofline

loop_site_4

Read

310for (int i = 0; i < ARRAY_SIZE_2 / 2; i++)

311{

312AoSoA_X[(j * (ARRAY_SIZE_2 / 2)) + i] = AoSoA_Y[j].a[i] + AoSoA_Y[j].a[i]

313+ AoSoA_Y[j].b[i] + AoSoA_Y[j].b[i];

314}

+P2

1

Unit stride

roofline.cpp:312

AoSoA_X

240B

libc.so.6; roofline

loop_site_4

Write

+P3

1

Unit stride

roofline.cpp:313

AoSoA_Y

240B

libc.so.6; roofline

loop_site_4

Read

+P4

Parallel site information

roofline.cpp:310

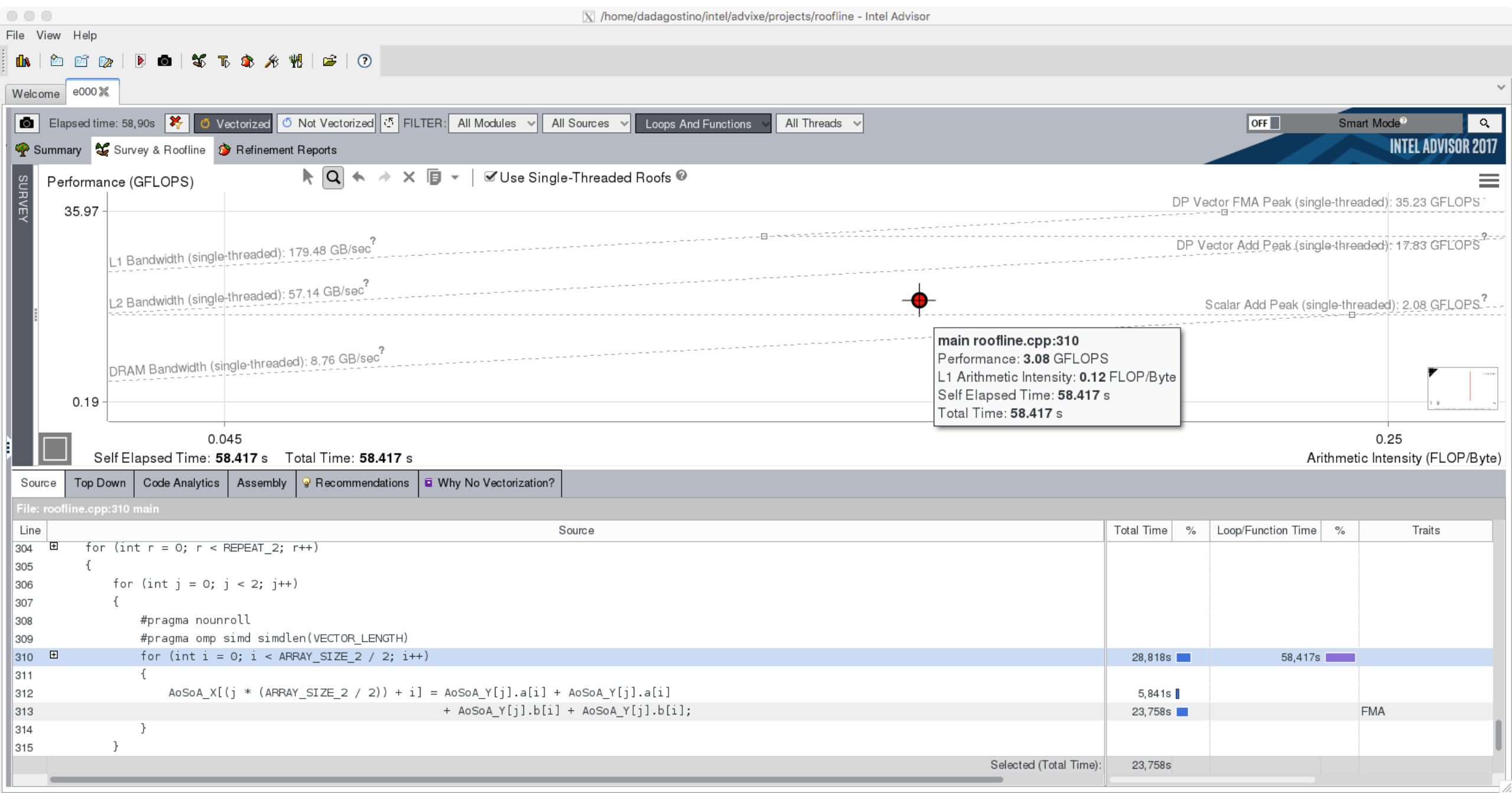
roofline

loop_site_4

INTEL ADVISOR 2017

Very time consuming!

icc -g -O2 -std=c++11 -qopenmp -xHost -qopt-report=5 -qopt-report-phase=vec roofline.cpp -o roofline



For the others cases follow the video tutorial

<https://software.intel.com/en-us/videos/roofline-analysis-in-intel-advisor-2017>

<https://www.youtube.com/watch?v=h2QEM1HpFgg>

Add efficiency to MatVector

Intel Advisor 2017 interface showing a performance report for a project named "MatVector". The interface includes a menu bar, a toolbar, and a main content area with a "Summary" tab selected.

A warning message states: **Higher instruction set architecture (ISA) available**. Consider recompiling your application using a higher ISA.

The main table displays performance data for various function call sites and loops. The columns include: Function Call Sites and Loops, Vector Issues, Self Time, Total Time, Type, Why No Vectorization?, Vectorized Loops, FLOPS And AVX-512 Mask Usage, and Trip Counts.

Function Call Sites and Loops	Vector Issues	Self Time	Total Time	Type	Why No Vectorization?	Vectorized Loops			FLOPS And AVX-512 Mask Usage			Trip Counts		Ins
						Vecto...	Gain ...	VL (V...	Self GFLOPS	Self AI	Mask Utilization	Average	Call Count	
[loop in matvec at Multiply.c:55]	2 Assumed dep...	47,638s	47,638s	Scalar	vector dependence pre...				0,424	0,083		50	101000000	
[loop in matvec at Multiply.c:45]	1 Opportunity for...	3,720s	51,358s	Scalar	outer loop was not auto...				0,054	0,042		101	1000000	
f matvec		0,020s	51,378s	Function									1000000	
f _start		0,000s	51,378s	Function									1	
[loop in main at Driver.c:145]		0,000s	51,378s	Scalar	loop with function call n...					0,031		1000000	1	
f main		0,000s	51,378s	Function						0,021			1	Shi

The "Recommendations" tab is selected, showing a section titled "Vector dependence prevents vectorization".

Cause: The compiler detected or assumed a vector dependence in the loop.

C++ Example:

```
int foo(float *A, int n) {
    int inx = 0;
    float max = A[0];
    int i;
    for (i=0; i < n; i++) {
        if (max < A[i]) {
            max = A[i];
            inx = i*i;
        }
    }
    return inx;
}
```

Fortran Example:

```
integer function foo(a, n)
    implicit none
    integer, intent(in) :: n
    real, intent(inout) :: a(n)
```

The Process for Vectorizing Your Application

- Measure Baseline Release Build Performance
(essentially compile the code without optimization and `-g`)
- Determine Hotspots
(with a profiler or manually instrumenting the code)
- Get Advice
(using compiler's features, Intel Advisor...)
- Implement Vectorization Recommendations
- Repeat!

<https://software.intel.com/en-us/articles/vectorization-toolkit/>